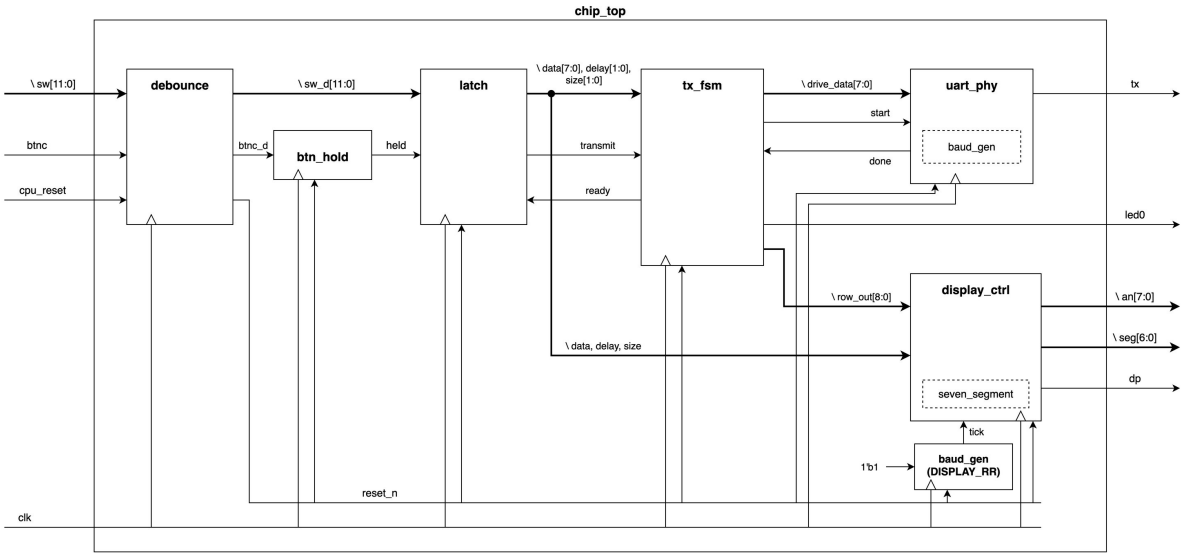
Lab 4: A TX UART Controller

Yuval Zecharia

16/10/2026

1. תהליך התכנון

1.1. ארכיטקטורה



איור 1: מבנה chip_top: תת-המודולים והאותות ביניהם.

1.2. ממשקי המודולים

טבלה 1: ממשק chip_top.

Port name	Direction	Type	Description
clk	Input	Wire	100 MHz system clock (E3)
cpu_reset	Input	Wire	CPU_RESETN push-button, active low
btnc	Input	Wire	Center push-button: 1 s hold latches SW and starts a run
sw	Input	Wire [11:0]	{SW14, SW13, SW9..SW0} = {size, delay, data}
tx	Output	Wire	UART serial line to the PC (UART_RXD_OUT, D4)
led0	Output	Wire	LED[0], toggles on every data byte
an	Output	Wire [7:0]	Digit anodes, active low, one digit on at a time
seg	Output	Wire [6:0]	Cathodes {CA..CG} of the active digit, active low
dp	Output	Wire	Decimal point, on (0) while AN3 is active

טבלה 2: הפרמטרים של chip_top וערכיהם בחומרה ובסימולציה.

Parameter	Hardware value	Simulation (tb_chip_top_0)	Meaning
BAUD_RATE	57600	10,000,000	uart_phy bit rate – equation (1)
DISPLAY_RR	4000	10,000,000	Digit tick of the display – equation (5)
MAX_COUNT_DEBOUNCE	2,000,000	4	Stable time of the debounce (20 ms) – equation (6)
MAX_COUNT_BTN_HOLD	100,000,000	10 (100 ns)	BTNC hold time (1 s) – equation (7)
DELAY50	5,000,000	20	Clocks of the 50 ms delay – equation (8)

טבלה 3: ממשק debounce.

Port name	Direction	Type	Description
clk	Input	Wire	Clock input from chip
cpu_reset	Input	Wire	Raw CPU_RESETN (asynchronous)
btnc	Input	Wire	Raw center button (asynchronous)
sw	Input	Wire [11:0]	Raw switches (asynchronous)
sw_d	Output	Wire [11:0]	Synchronized, debounced switches
btnc_d	Output	Wire	Synchronized, debounced button
cpu_reset_d	Output	Wire	reset_n for the whole design; INIT 0 gives a power-on reset

טבלה 4: ממשק btn_hold.

Port name	Direction	Type	Description
clk	Input	Wire	Clock input from chip
reset_n	Input	Wire	Synchronous reset (active low)
btn	Input	Wire	btnc_d from debounce
held	Output	Wire	One-clock pulse when the button was held for MAX_COUNT clocks

טבלה 5: ממשק latch.

Port name	Direction	Type	Description
clk	Input	Wire	Clock input from chip
reset_n	Input	Wire	Synchronous reset (active low)
ready	Input	Wire	tx_fsm is in IDLE
sw_d	Input	Wire [11:0]	Debounced switches
held	Input	Wire	1 s pulse from btn_hold
data / delay / size	Output	Reg [7:0] / [1:0] / [1:0]	Latched configuration, kept for the whole run
transmit	Output	Reg	One-clock start of a run, set on the same edge that latches the configuration

טבלה 6: ממשק tx_fsm.

Port name	Direction	Type	Description
clk	Input	Wire	Clock input from chip
reset_n	Input	Wire	Synchronous reset (active low)
data / delay / size	Input	Wire [7:0] / [1:0] / [1:0]	Configuration from latch
transmit	Input	Wire	Starts a run (IDLE only)
done	Input	Wire	uart_phy finished the current byte
drive_data	Output	Reg [7:0]	Byte for uart_phy: data, 0x20, 0x0D or 0x0A
start	Output	Reg	One-clock request to uart_phy
led0	Output	Reg	Toggles on every data byte
row_out	Output	Wire [8:0]	Rows transmitted, 0..256
ready	Output	Wire	state == IDLE

טבלה 7: ממשק uart_phy.

Port name	Direction	Type	Description
clk	Input	Wire	Clock input from chip
reset_n	Input	Wire	Synchronous reset (active low)
data	Input	Wire [7:0]	Byte to transmit
start	Input	Wire	Loads {1, data, 0} in IDLE
tx	Output	Wire	Serial line, LSB first; 1 when idle
done	Output	Wire	One-clock pulse at the end of the stop bit

טבלה 8: ממשק baud_gen.

Port name	Direction	Type	Description
clk	Input	Wire	Clock input from chip
reset_n	Input	Wire	Synchronous reset (active low)
en	Input	Wire	Counter runs while 1 and restarts at 0 when 0
baud_tick	Output	Wire	One-clock tick every 100 MHz / BAUD_RATE clocks

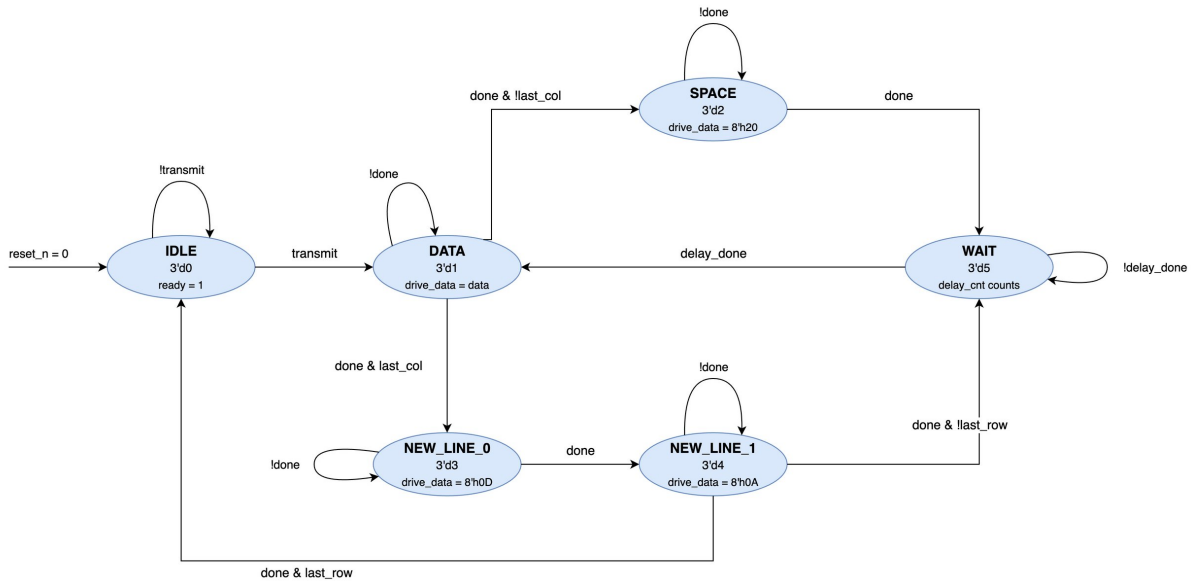
טבלה 9: ממשק display_ctrl.

Port name	Direction	Type	Description
clk	Input	Wire	Clock input from chip
reset_n	Input	Wire	Synchronous reset (active low)
tick	Input	Wire	Digit tick – advances the anode ring
data / delay / size	Input	Wire [7:0] / [1:0] / [1:0]	Values for T0, T1, T2
row	Input	Wire [8:0]	row_out for T3 (FF when row[8] = 1)
an	Output	Reg [7:0]	Anode ring, starts at 8'hFE
seg	Output	Wire [6:0]	Cathodes from seven_segment
dp	Output	Wire	an[3]: decimal point of T1

טבלה 10: ממשק seven_segment.

Port name	Direction	Type	Description
dig_in	Input	Wire [3:0]	Hex digit 0–F
ss_dig	Output	Reg [6:0]	Active-low cathodes {CA..CG}

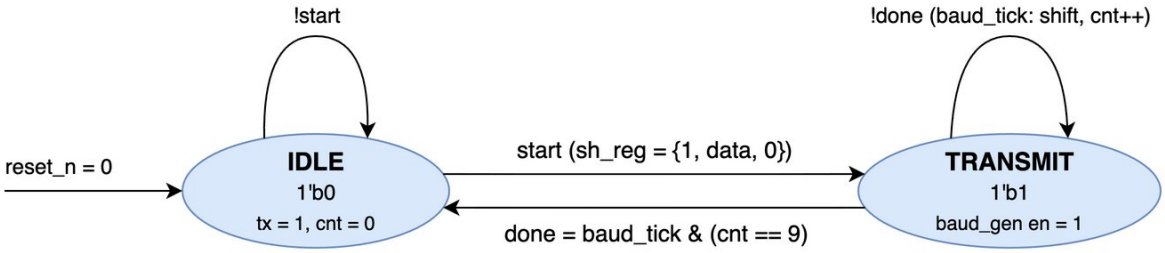
1.3. מכונות המצבים



איור 2: דיאגרמת המצבים של tx_fsm.

טבלה 11: טבלת המעברים של tx_fsm.

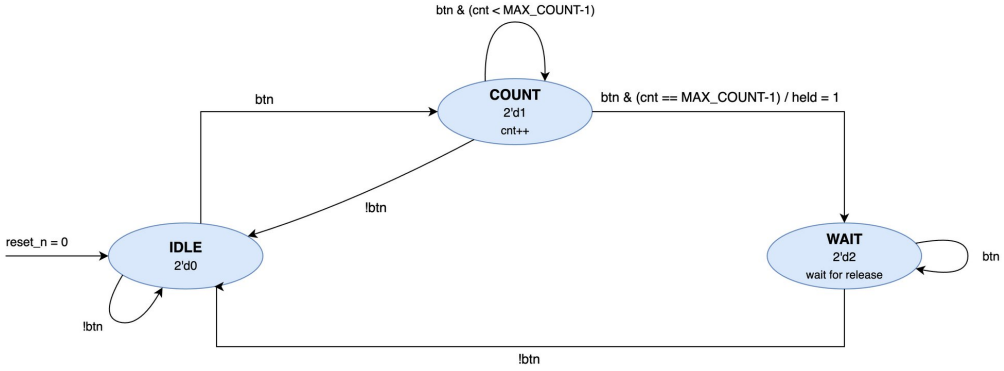
Current state	Condition	Next state	Registered on the transition
IDLE (0)	transmit	DATA	start = 1, drive_data = data, led0 toggles; col = row = 0
DATA (1)	done & !last_col	SPACE	start = 1, drive_data = 8'h20
DATA (1)	done & last_col	NEW_LINE_0	start = 1, drive_data = 8'h0D
SPACE (2)	done	WAIT	col++, delay_cnt = 0
NEW_LINE_0 (3)	done	NEW_LINE_1	start = 1, drive_data = 8'h0A
NEW_LINE_1 (4)	done & !last_row	WAIT	row++, col = 0, delay_cnt = 0
NEW_LINE_1 (4)	done & last_row	IDLE	row++ (row_out = N)
WAIT (5)	delay_done	DATA	start = 1, drive_data = data, led0 toggles



איור 3: דיאגרמת המצבים של uart_phy.

טבלה 12: טבלת המעברים של uart_phy.

Current state	Condition	Next state	Registered on the transition
IDLE (0)	start	TRANSMIT	sh_reg = {1, data, 0}: tx = 0 (start bit)
TRANSMIT (1)	baud_tick & cnt < 9	TRANSMIT	sh_reg = {1, sh_reg[9:1]}, cnt++
TRANSMIT (1)	done = baud_tick & cnt == 9	IDLE	sh_reg = all 1, cnt = 0



איור 4: דיאגרמת המצבים של btn_hold.

טבלה 13: טבלת המעברים של btn_hold.

Current state	Condition	Next state	Registered on the transition
IDLE (0)	btn	COUNT	cnt = 0
COUNT (1)	btn & cnt < MAX_COUNT-1	COUNT	cnt++
COUNT (1)	!btn	IDLE	cnt = 0
COUNT (1)	btn & cnt == MAX_COUNT-1	WAIT	cnt = 0; held = 1 in this clock
WAIT (2)	!btn	IDLE	–

1.4. שיקולי תכנון

- קצב השידור: baud_gen סופר עד COUNT ומפיק baud_tick רק כשה-en שלו פעיל, כלומר במצב TRANSMIT. COUNT מחושב בחלוקה שלמה, ולכן הקצב בפועל מהיר מעט מ-57600 (משוואה 1), בשגיאה יחסית של 0.0064% (משוואה 2). המקלט דוגם כל ביט באמצעו, והשגיאה מצטברת לאורך הפריים עד ה-stop bit, שנדגם 9.5 ביטים אחרי הירידה של ה-start bit. שם ההסטה היא כ-0.06% מביט (משוואה 3), הרבה מתחת לחצי ביט, ההסטה שבה המקלט כבר היה דוגם ביט שגוי. בסימולציה COUNT=10 בדיוק ואין שגיאה. פריים של 10 ביטים נמשך 173.6µs (משוואה 4).

$$COUNT = \lfloor \frac{100\ MHz}{57600} \rfloor = \lfloor 1736.11 \rfloor = 1736,\ f_{actual} = \frac{100\ MHz}{1736} = 57603.69\ bps \quad (1)$$

$$error = \frac{f_{actual} - BAUD_RATE}{BAUD_RATE} = \frac{57603.69 - 57600}{57600} = 6.40 \cdot 10^{-5} = +0.0064\% \quad (2)$$

$$error_{stop\ bit} = 9.5 \cdot error = 9.5 \cdot 0.0064\% \approx 0.061\%\ T_{bit} \quad (3)$$

$$T_{frame} = 10 \cdot COUNT \cdot T_{clk} = 10 \cdot 1736 \cdot 10\ ns = 173.6\ \mu s \quad (4)$$

- תצוגה: מופע שני של baud_gen עם DISPLAY_RR=4000 מפיק tick של 4kHz, והטבעת של 8 ספרות מקבלת רענון של 500Hz (משוואה 5). ה-tick משמש כ-enable ולא כשעון מחולק, ולכן אין תחום שעון נוסף. dp מחובר ל-an[3] ודולק רק בספרה השמאלית של T1, ולכן T1 מציג את ההשהיה ביחידות של 0.0, 0.5, 1.0 :100ms או 2.0.

$$f_{tick} = \frac{f_{clk}}{25000} = 4\ kHz,\ f_{refresh} = \frac{f_{tick}}{8} = 500\ Hz \quad (5)$$

- debounce וסנכרון: כל כניסה אסינכרונית עוברת שני דלגלגים, גלאי שינוי ומונה יציבות של 20ms (משוואה 6). ל-CPU_RESETN מונה נפרד עם ערכי INIT, ולכן n_reset יוצא 0 אחרי טעינת ה-bitstream (power-on reset). המתגים והכפתור הם וקטור אחד של 13 ביטים עם מונה משותף.

$$MAX_COUNT_DEBOUNCE = f_{clk} \cdot 20\ ms = 2 \cdot 10^6 \ \Rightarrow\ \lceil log_2(2 \cdot 10^6) \rceil = 21\ bit \quad (6)$$

- החזקה של שנייה: btn_hold סופר MAX_COUNT מחזורים כל עוד btnc_d=1 (משוואה 7), ומפיק held אחד בכל לחיצה. המצב WAIT מחכה לשחרור, ולכן החזקה ארוכה אינה מפעילה שידור נוסף.

$$MAX_COUNT_BTN_HOLD = f_{clk} \cdot 1\ s = 10^8 \ \Rightarrow\ \lceil log_2 10^8 \rceil = 27\ bit \quad (7)$$

- השהיה בין בתים: delay מפוענח ל-DELAY100, DELAY50, 0 או DELAY200, ורוחב delay_cnt נקבע לפי הערך הגדול (משוואה 8). ב-delay=0 התנאי delay_done מתקיים מיד, ומצב WAIT נמשך מחזור אחד.

$$DELAY200 = 4 \cdot f_{clk} \cdot 50\ ms = 2 \cdot 10^7 \ \Rightarrow\ \lceil log_2(DELAY200 + 1) \rceil = 25\ bit \quad (8)$$

- תווים מיוחדים ומונים: col סופר בתי נתון בשורה ו-row סופר שורות. אחרי בית שאינו אחרון בשורה נשלח 0x20, ואחרי האחרון נשלחים 0D ו-0A, גם בשורה האחרונה. מכאן מספר הבתים בריצה (משוואה 9). אחרי השורה האחרונה אין WAIT.

$$N_{bytes} = N \cdot (N + (N-1) + 2) = N \cdot (2N + 1) \ \Rightarrow\ N = 32:\ 2080,\ N = 256:\ 131328 \quad (9)$$

- מונה השידור: מדרישות המעבדה עולה כי T3 מציג את מספר השורות שנשלחו, ובמקום אחר כי מונה בתי הנתון מוצג בו. מספר הבתים מגיע עד 65536 ואינו נכנס בשתי ספרות hex, ולכן T3 מציג את row_out, שעולה פעם אחת בכל N בתי נתון. row ברוחב 9 ביטים כדי להכיל 256. שתי ספרות hex מגיעות רק עד FF, ולכן T2 מציג FF עבור N=256, ו-T3 מציג FF כש-row[8]=1, כלומר כש-row מגיע ל-256. LED0 מתהפך רק בבתי נתון, כמו מונה בתי הנתון, ולא בתווים 0x0D ,0x20 ו-0x0A.
- latch: התצורה נדגמת רק כש-(load_cfg = held & ready) ready=1, כך שלחיצה באמצע ריצה אינה משנה אותה. transmit הוא load_cfg מושהה במחזור אחד, ולכן tx_fsm קורא כבר את הערכים החדשים.
- אילוצי תזמון: הכניסות מסונכרנות בתוך העיצוב, והיציאות (LED, תצוגה ו-UART ב-57600bps) איטיות ביחס לשעון. לכן הוגדרו להן set_false_path בקובץ ה-XDC במקום input/output delay, וכך דוח ה-Methodology נקי מאזהרות TIMING-18.

- סימולציה: הפרמטרים מוקטנים רק בטסטבנץ' (טבלה 2), וערכי החומרה נשארים בקוד. לחיצה מגיעה ל-held אחרי 17 מחזורים: 2 מחזורי סנכרון, מחזור של גלאי השינוי, MAX_COUNT_DEBOUNCE−1 מחזורי יציבות ומחזור של רגיסטר היציאה, ואחריהם מחזור כניסה ל-COUNT ו-MAX_COUNT_BTN_HOLD−1 מחזורי ספירה (משוואה 10). held מתקבל רק מלחיצה של 11 מחזורים לפחות (משוואה 11). ה-debounce מזיז את שני הקצוות של הלחיצה באותה כמות ולכן אינו משנה את הרוחב הנדרש.

$$t_{held} = (2 + 1 + (4-1) + 1) + (1 + (10-1)) = 7 + 10 = 17\ clk \quad (10)$$

$$L_{min} = MAX_COUNT_BTN_HOLD + 1 = 11\ clk \quad (11)$$

1.5. קוד המקור

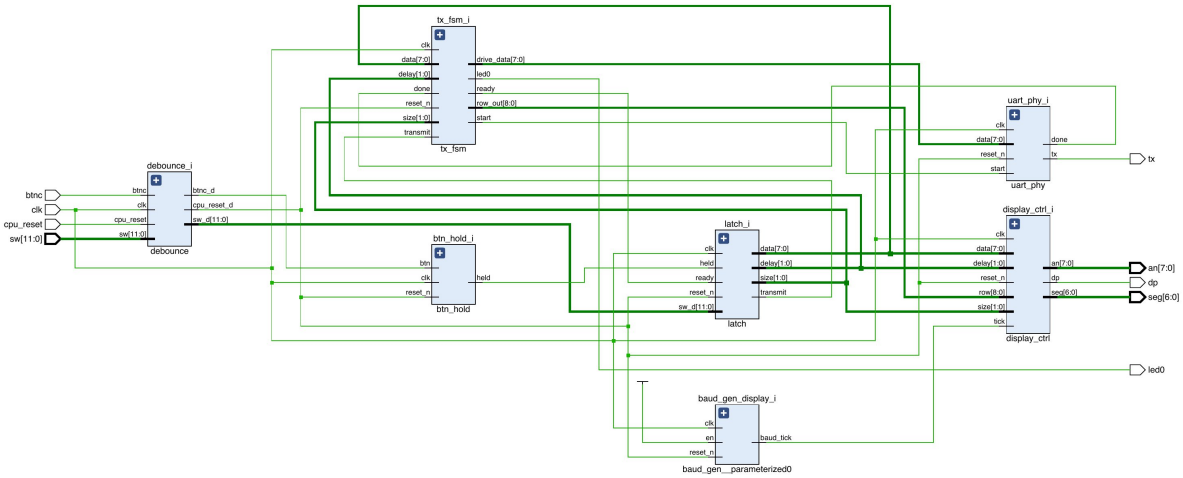
קוד המקור של תשעת המודולים ושורות האילוצים הפעילות מקובץ ה-XDC מובאים בנספח א'.

2. בדיקת Lint

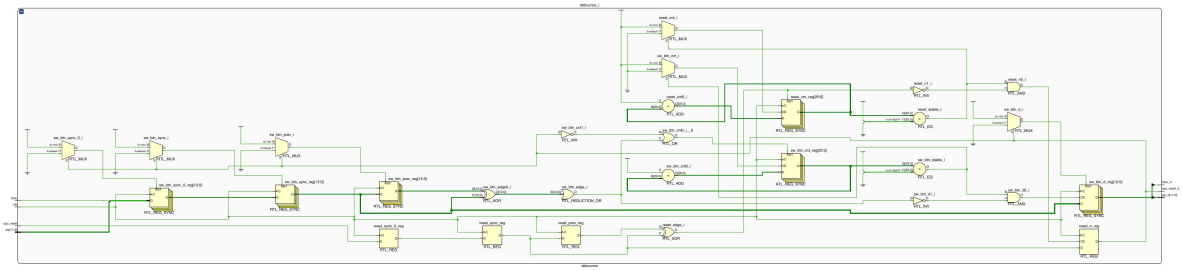
טבלה 14: תוצאות בדיקות ה-Lint וה-Elaborated Design.

Check	Scope	Result	Report file
RTL Linter (Run Linter)	chip_top + all sub-modules	0 violations	vivado_lint_report_chip_top.txt
Elaborated design – Report DRC	chip_top (397 instances)	0 checks	elaborated_design_drc.rpt
Elaborated design – Report Methodology	chip_top	1 advisory: RFFI-1 (an_reg[3] drives an[3] and dp)	elaborated_design_methodology.rpt

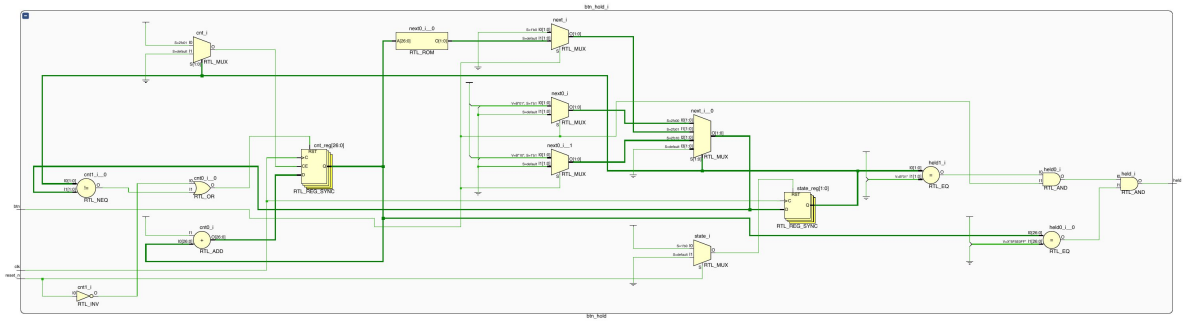
2.1. סכמות RTL (Elaborated Design)



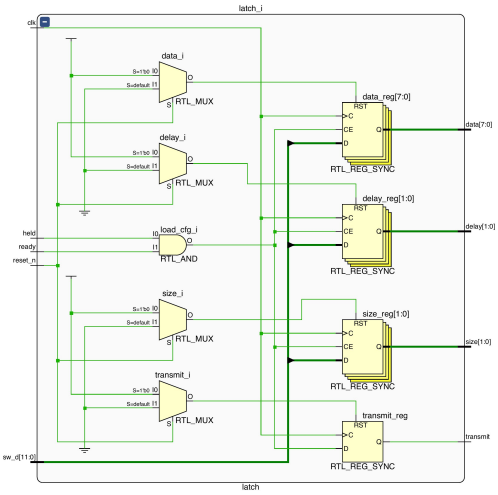
איור 5: סכמת ה-RTL של chip_top: כל תת-המודולים ו-baud_gen של התצוגה.



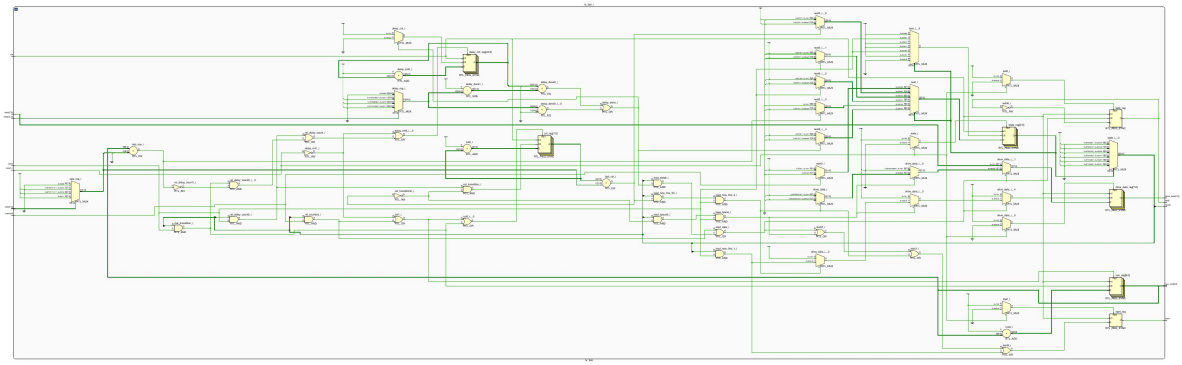
איור 6: סכמת ה-RTL של debounce: סנכרון, גלאי שינוי, מונה יציבות ורגיסטר יציאה לכל אחד משני הענפים (reset ו-{SW, BTNC}).



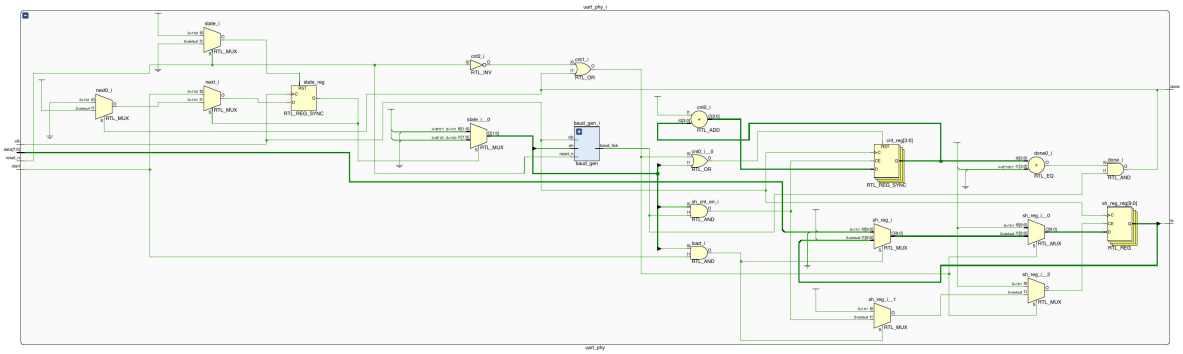
איור 7: סכמת ה-RTL של btn_hold: רגיסטר המצב, לוגיקת המצב הבא, המונה וה-AND של held.



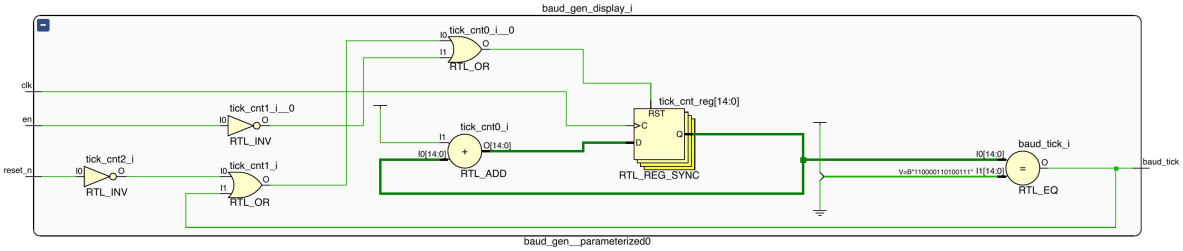
איור 8: סכמת ה-RTL של latch: רגיסטר התצורה עם enable של load_cfg ודלגלג transmit.



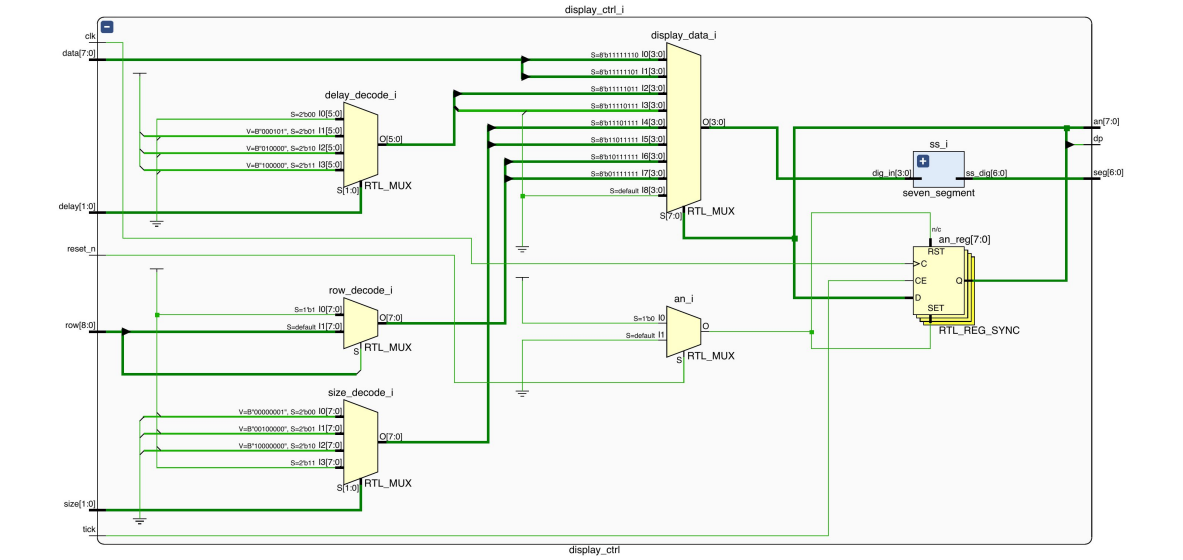
איור 9: סכמת ה-RTL של tx_fsm: מפענחי delay ו-size, המונים col, row ו-delay_cnt, מכונת המצבים ורגיסטרי היציאה.



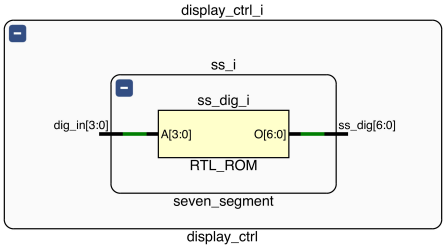
איור 10: סכמת ה-RTL של uart_phy: baud_gen, מונה הביטים, רגיסטר ההזזה ורגיסטר המצב.



איור 11: סכמת ה-RTL של baud_gen: מונה, משווה ואיפוס.



איור 12: סכמת ה-RTL של display_ctrl: טבעת האנודות, המפענחים של T1–T3, הבורר ו-seven_segment.



איור 13: סכמת ה-RTL של seven_segment: משפט ה-case כ-ROM.

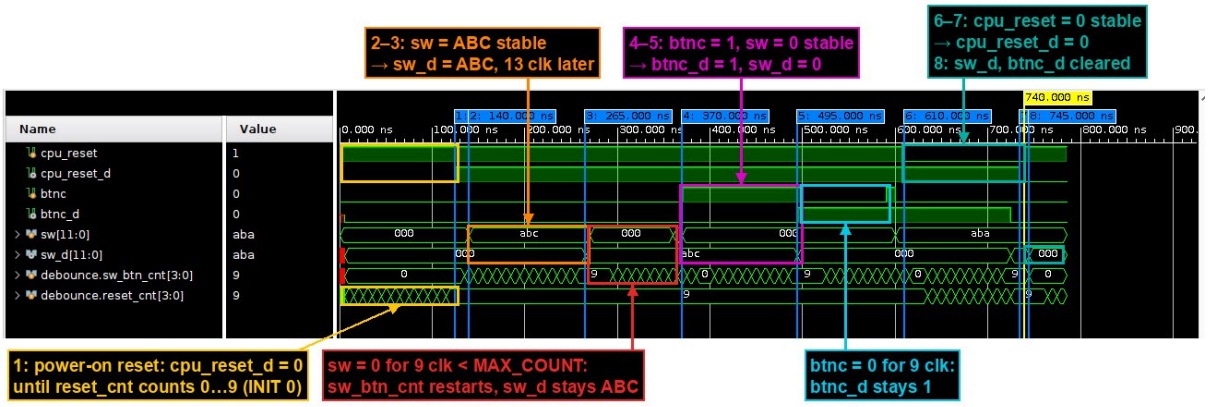
3. תוצאות סימולציה

3.1. בדיקת debounce – tb_debounce

פלט הטסטבנץ' (tb_debounce_xsim.log):

```
Time: 0 | cpu_reset=1 | cpu_reset_d=0 | btnc=0 | btnc_d=x | sw=0| sw_d=xxx
Time: 5 | cpu_reset=1 | cpu_reset_d=0 | btnc=0 | btnc_d=0 | sw=0| sw_d=0
Time: 125 | cpu_reset=1 | cpu_reset_d=1 | btnc=0 | btnc_d=0 | sw=0| sw_d=0
Time: 140 | cpu_reset=1 | cpu_reset_d=1 | btnc=0 | btnc_d=0 | sw=abc| sw_d=0
Time: 265 | cpu_reset=1 | cpu_reset_d=1 | btnc=0 | btnc_d=0 | sw=abc| sw_d=abc
Time: 270 | cpu_reset=1 | cpu_reset_d=1 | btnc=0 | btnc_d=0 | sw=0| sw_d=abc
Time: 360 | cpu_reset=1 | cpu_reset_d=1 | btnc=0 | btnc_d=0 | sw=abc| sw_d=abc
Time: 370 | cpu_reset=1 | cpu_reset_d=1 | btnc=1 | btnc_d=0 | sw=0| sw_d=abc
Time: 495 | cpu_reset=1 | cpu_reset_d=1 | btnc=1 | btnc_d=1 | sw=0| sw_d=0
Time: 500 | cpu_reset=1 | cpu_reset_d=1 | btnc=0 | btnc_d=1 | sw=0| sw_d=0
Time: 590 | cpu_reset=1 | cpu_reset_d=1 | btnc=1 | btnc_d=1 | sw=0| sw_d=0
Time: 600 | cpu_reset=1 | cpu_reset_d=1 | btnc=0 | btnc_d=1 | sw=aba| sw_d=0
Time: 610 | cpu_reset=0 | cpu_reset_d=1 | btnc=0 | btnc_d=1 | sw=aba| sw_d=0
Time: 725 | cpu_reset=0 | cpu_reset_d=1 | btnc=0 | btnc_d=0 | sw=aba| sw_d=aba
Time: 735 | cpu_reset=0 | cpu_reset_d=0 | btnc=0 | btnc_d=0 | sw=aba| sw_d=aba
Time: 740 | cpu_reset=1 | cpu_reset_d=0 | btnc=0 | btnc_d=0 | sw=aba| sw_d=aba
Time: 745 | cpu_reset=1 | cpu_reset_d=0 | btnc=0 | btnc_d=0 | sw=aba| sw_d=0
$finish called at time : 785 ns : File "sim/tb_debounce.v" Line 53
```

צורות הגל מוצגות באיור 14.



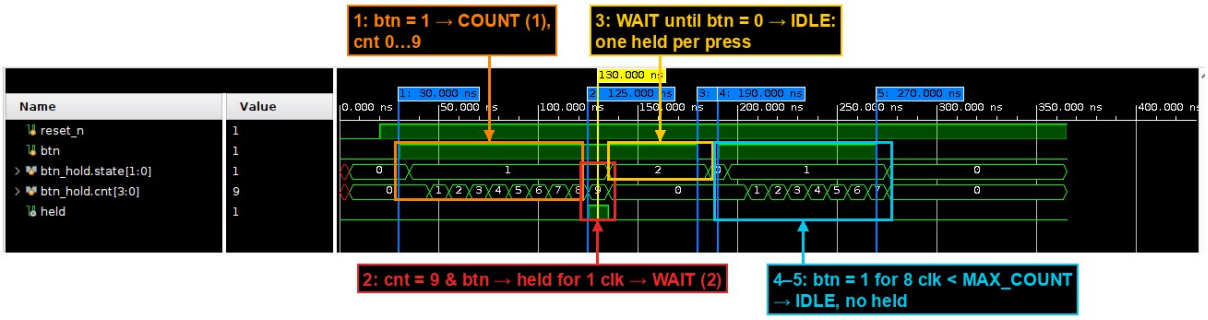
איור 14: tb_debounce – כל הריצה: סנכרון ו-debounce של cpu_reset, btnc ו-sw.

3.2. בדיקת btn_hold – tb_btn_hold

פלט הטסטבנץ' (tb_btn_hold_xsim.log):

```
Time: 0 | btn=0 | held=0
Time: 30 | btn=1 | held=0
Time: 125 | btn=1 | held=1
Time: 135 | btn=1 | held=0
Time: 180 | btn=0 | held=0
Time: 190 | btn=1 | held=0
Time: 270 | btn=0 | held=0
$finish called at time : 365 ns : File "sim/tb_btn_hold.v" Line 31
```

צורות הגל מוצגות באיור 15.



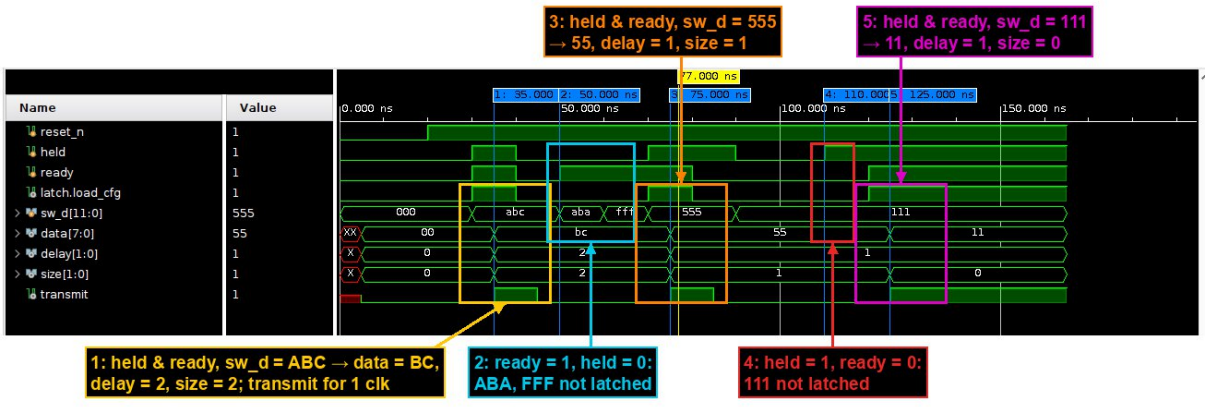
איור 15: tb_btn_hold – כל הריצה: לחיצה ארוכה ולחיצה קצרה.

3.3. tb_latch – בדיקת latch

פלט הטסטבנץ' (tb_latch_xsim.log):

```
Time: 0 | held=0 | sw_d=0 | ready=0 | data=xx | delay=x | size=x | transmit=x
Time: 5 | held=0 | sw_d=0 | ready=0 | data=0 | delay=0 | size=0 | transmit=0
Time: 30 | held=1 | sw_d=abc | ready=1 | data=0 | delay=0 | size=0 | transmit=0
Time: 35 | held=1 | sw_d=abc | ready=1 | data=bc | delay=2 | size=2 | transmit=1
Time: 40 | held=0 | sw_d=abc | ready=0 | data=bc | delay=2 | size=2 | transmit=1
Time: 45 | held=0 | sw_d=abc | ready=0 | data=bc | delay=2 | size=2 | transmit=0
Time: 50 | held=0 | sw_d=aba | ready=1 | data=bc | delay=2 | size=2 | transmit=0
Time: 60 | held=0 | sw_d=fff | ready=1 | data=bc | delay=2 | size=2 | transmit=0
Time: 70 | held=1 | sw_d=555 | ready=1 | data=bc | delay=2 | size=2 | transmit=0
Time: 75 | held=1 | sw_d=555 | ready=1 | data=55 | delay=1 | size=1 | transmit=1
Time: 80 | held=1 | sw_d=555 | ready=0 | data=55 | delay=1 | size=1 | transmit=1
Time: 85 | held=1 | sw_d=555 | ready=0 | data=55 | delay=1 | size=1 | transmit=0
Time: 90 | held=0 | sw_d=111 | ready=0 | data=55 | delay=1 | size=1 | transmit=0
Time: 110 | held=1 | sw_d=111 | ready=0 | data=55 | delay=1 | size=1 | transmit=0
Time: 120 | held=1 | sw_d=111 | ready=1 | data=55 | delay=1 | size=1 | transmit=0
Time: 125 | held=1 | sw_d=111 | ready=1 | data=11 | delay=1 | size=0 | transmit=1
$finish called at time : 165 ns : File "sim/tb_latch.v" Line 41
```

צורות הגל מוצגות באיור 16.



איור 16: tb_latch – כל הריצה: דגימת התצורה ב-held & ready.

3.4. tb_tx_fsm_0 – בדיקת tx_fsm

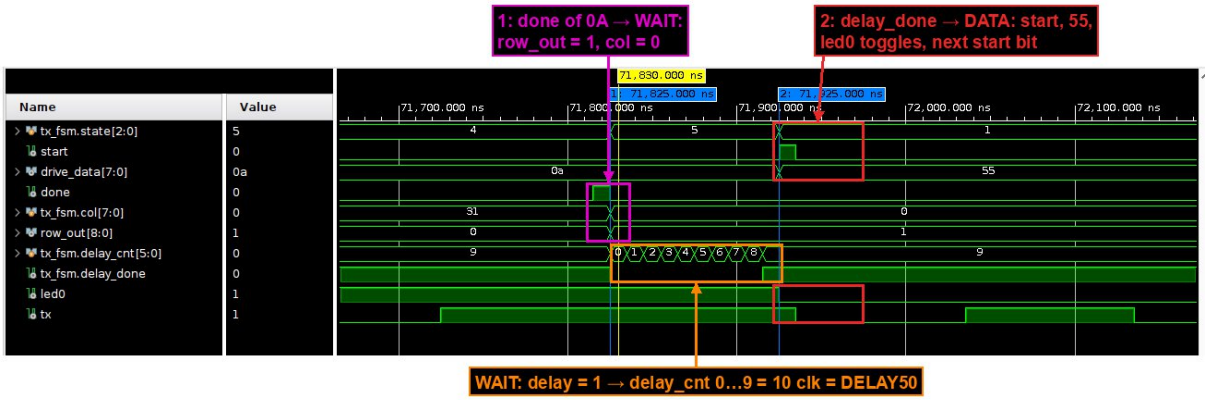
פלט הטסטבנץ' (tb_tx_fsm_0_xsim.log):

```
Time: 15 | data=0x41 | size=0 | delay=0
Time: 3065 | data=0x55 | delay=1 | size=1
Time: 2206185 | data=0xAA | delay=3 | size=1
$finish called at time : 4716205 ns : File "sim/tb_tx_fsm_0.v" Line 46
```

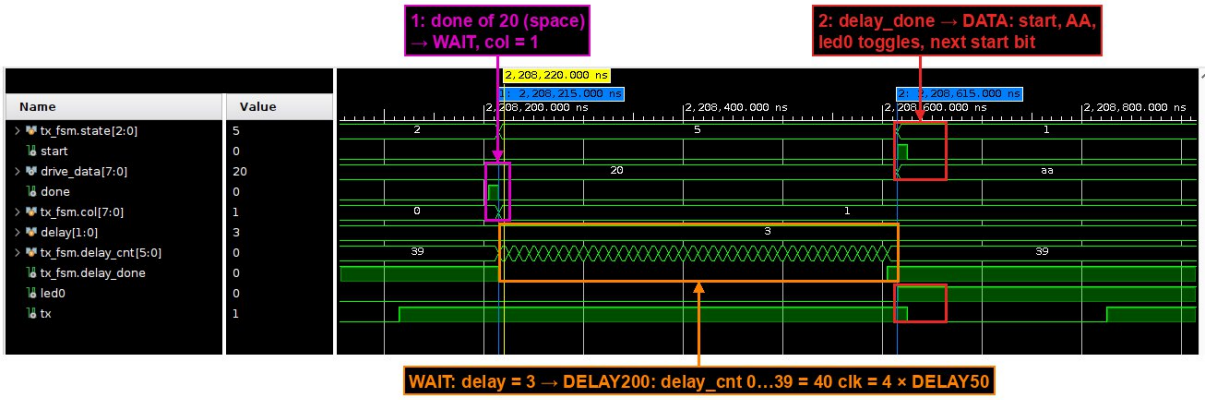
צורות הגל מוצגות באיורים 17–19.



איור 17: tb_tx_fsm_0 – זום: ריצת 1×1 של 0x41.



איור 18: tb_tx_fsm_0 – זום: סוף השורה הראשונה והמצב WAIT, 32×32 עם delay=1.



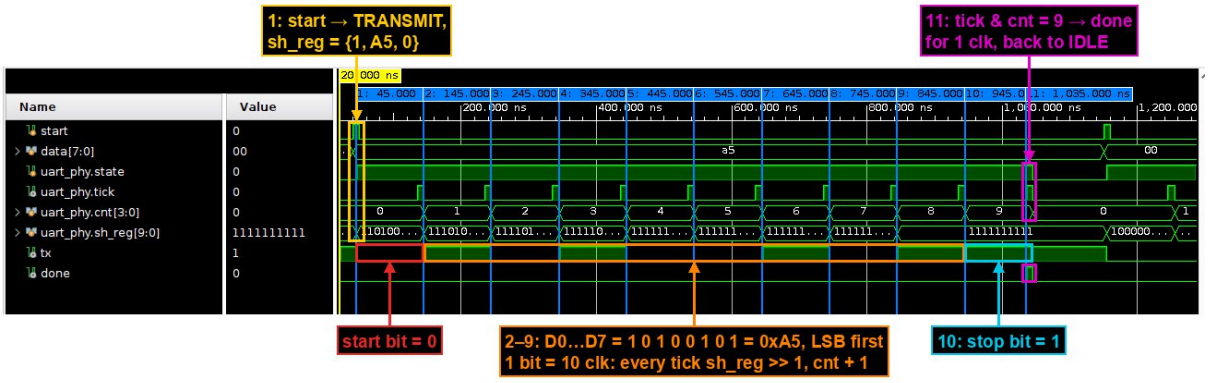
איור 19: tb_tx_fsm_0 – זום: רווח והמצב WAIT, 32×32 עם delay=3.

3.5. tb_uart_phy_0 – בדיקת uart_phy

פלט הטסטבנץ' (tb_uart_phy_0_xsim.log):

```
Time: 0 | data=0 | start=0 | tx=x | done=x
Time: 5 | data=0 | start=0 | tx=1 | done=0
Time: 40 | data=a5 | start=1 | tx=1 | done=0
Time: 45 | data=a5 | start=1 | tx=0 | done=0
Time: 50 | data=a5 | start=0 | tx=0 | done=0
Time: 145 | data=a5 | start=0 | tx=1 | done=0
Time: 245 | data=a5 | start=0 | tx=0 | done=0
Time: 345 | data=a5 | start=0 | tx=1 | done=0
Time: 445 | data=a5 | start=0 | tx=0 | done=0
Time: 645 | data=a5 | start=0 | tx=1 | done=0
Time: 745 | data=a5 | start=0 | tx=0 | done=0
Time: 845 | data=a5 | start=0 | tx=1 | done=0
Time: 1035 | data=a5 | start=0 | tx=1 | done=1
Time: 1045 | data=a5 | start=0 | tx=1 | done=0
   ... (12 more lines: frames 0x00 and 0xFF) ...
$finish called at time : 3365 ns : File "sim/tb_uart_phy_0.v" Line 46
```

צורות הגל מוצגות באיור 20.



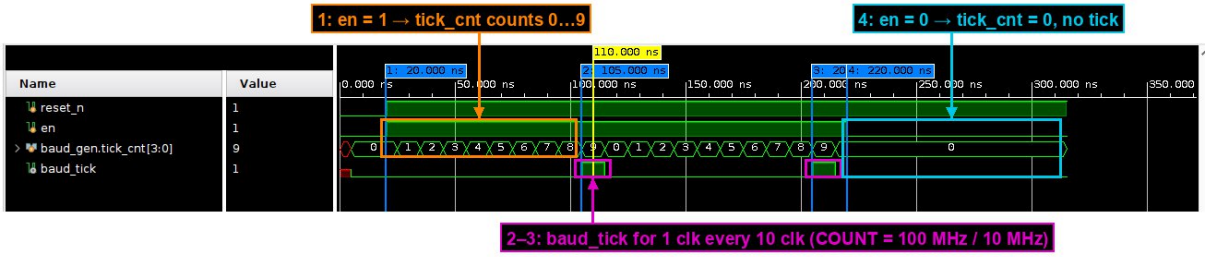
איור 20: tb_uart_phy_0 – זום: שידור הבית 0xA5.

3.6. בדיקת baud_gen – tb_baud_gen

פלט הטסטבנץ' (tb_baud_gen_xsim.log):

```
Time: 0 | en=0 | baud_tick=x
Time: 5 | en=0 | baud_tick=0
Time: 20 | en=1 | baud_tick=0
Time: 105 | en=1 | baud_tick=1
Time: 115 | en=1 | baud_tick=0
Time: 205 | en=1 | baud_tick=1
Time: 215 | en=1 | baud_tick=0
Time: 220 | en=0 | baud_tick=0
$finish called at time : 315 ns : File "sim/tb_baud_gen.v" Line 27
```

צורות הגל מוצגות באיור 21.



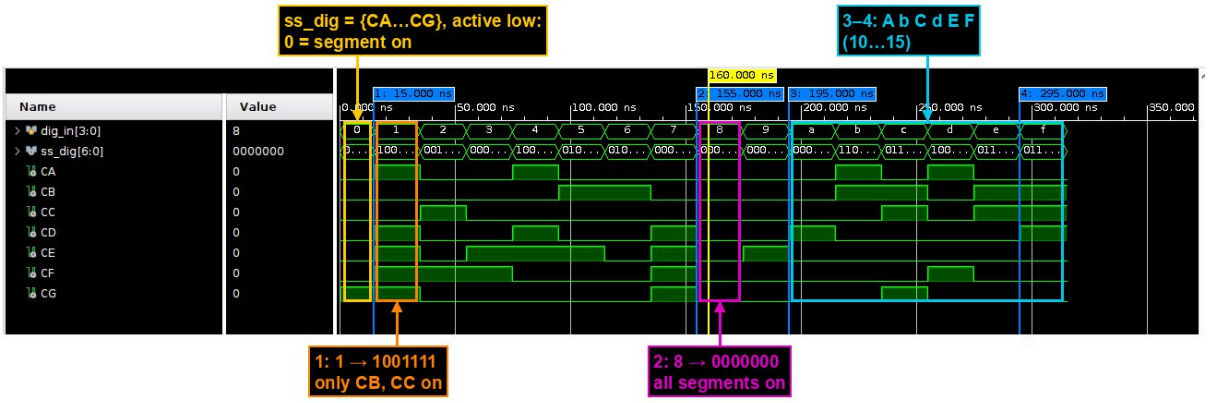
איור 21: tb_baud_gen – כל הריצה: baud_tick עם en=1 ועם en=0.

3.7. בדיקת seven_segment – tb_seven_segment

פלט הטסטבנץ' (tb_seven_segment_xsim.log):

```
Time: 0 | dig_in=0 | ss_dig = 0000001
Time: 15 | dig_in=1 | ss_dig = 1001111
Time: 35 | dig_in=2 | ss_dig = 0010010
Time: 55 | dig_in=3 | ss_dig = 0000110
Time: 75 | dig_in=4 | ss_dig = 1001100
Time: 95 | dig_in=5 | ss_dig = 0100100
Time: 115 | dig_in=6 | ss_dig = 0100000
Time: 135 | dig_in=7 | ss_dig = 0001111
Time: 155 | dig_in=8 | ss_dig = 0000000
Time: 175 | dig_in=9 | ss_dig = 0000100
Time: 195 | dig_in=a | ss_dig = 0001000
Time: 215 | dig_in=b | ss_dig = 1100000
Time: 235 | dig_in=c | ss_dig = 0110001
Time: 255 | dig_in=d | ss_dig = 1000010
Time: 275 | dig_in=e | ss_dig = 0110000
Time: 295 | dig_in=f | ss_dig = 0111000
$finish called at time : 315 ns : File "sim/tb_seven_segment.v" Line 21
```

צורות הגל מוצגות באיור 22.



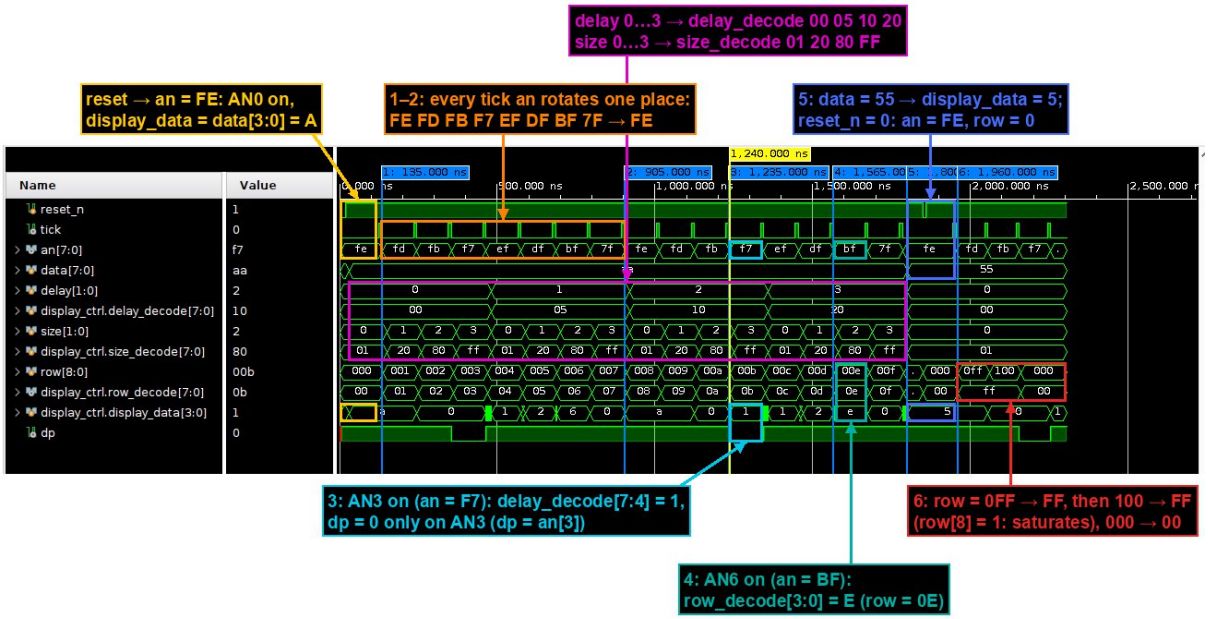
איור 22: tb_seven_segment – כל 16 הספרות.

3.8. tb_display_ctrl – בדיקת display_ctrl

פלט הטסטבנץ' (tb_display_ctrl_xsim.log):

```
Time: 0 | data=00 | delay=0 | size=0 | tick_en=0 | row=000000000 | an=xxxxxxxx | seg=0000001 | dp=x
Time: 5 | data=00 | delay=0 | size=0 | tick_en=0 | row=000000000 | an=11111110 | seg=0000001 | dp=1
Time: 30 | data=aa | delay=0 | size=0 | tick_en=0 | row=000000000 | an=11111110 | seg=0001000 | dp=1
Time: 40 | data=aa | delay=0 | size=0 | tick_en=1 | row=000000000 | an=11111110 | seg=0001000 | dp=1
Time: 135 | data=aa | delay=0 | size=0 | tick_en=1 | row=000000000 | an=11111101 | seg=0001000 | dp=1
Time: 140 | data=aa | delay=0 | size=0 | tick_en=0 | row=000000001 | an=11111101 | seg=0001000 | dp=1
Time: 150 | data=aa | delay=0 | size=1 | tick_en=1 | row=000000001 | an=11111101 | seg=0001000 | dp=1
Time: 245 | data=aa | delay=0 | size=1 | tick_en=1 | row=000000001 | an=11111011 | seg=0000001 | dp=1
Time: 250 | data=aa | delay=0 | size=1 | tick_en=0 | row=000000010 | an=11111011 | seg=0000001 | dp=1
Time: 260 | data=aa | delay=0 | size=2 | tick_en=1 | row=000000010 | an=11111011 | seg=0000001 | dp=1
Time: 355 | data=aa | delay=0 | size=2 | tick_en=1 | row=000000010 | an=11110111 | seg=0000001 | dp=0
Time: 360 | data=aa | delay=0 | size=2 | tick_en=0 | row=000000011 | an=11110111 | seg=0000001 | dp=0
Time: 370 | data=aa | delay=0 | size=3 | tick_en=1 | row=000000011 | an=11110111 | seg=0000001 | dp=0
Time: 465 | data=aa | delay=0 | size=3 | tick_en=1 | row=000000011 | an=11101111 | seg=0111000 | dp=1
Time: 470 | data=aa | delay=0 | size=3 | tick_en=0 | row=000000100 | an=11101111 | seg=0111000 | dp=1
Time: 480 | data=aa | delay=1 | size=0 | tick_en=1 | row=000000100 | an=11101111 | seg=1001111 | dp=1
Time: 575 | data=aa | delay=1 | size=0 | tick_en=1 | row=000000100 | an=11011111 | seg=0000001 | dp=1
Time: 580 | data=aa | delay=1 | size=0 | tick_en=0 | row=000000101 | an=11011111 | seg=0000001 | dp=1
Time: 590 | data=aa | delay=1 | size=1 | tick_en=1 | row=000000101 | an=11011111 | seg=0010010 | dp=1
Time: 685 | data=aa | delay=1 | size=1 | tick_en=1 | row=000000101 | an=10111111 | seg=0100100 | dp=1
Time: 690 | data=aa | delay=1 | size=1 | tick_en=0 | row=000000110 | an=10111111 | seg=0100000 | dp=1
Time: 700 | data=aa | delay=1 | size=2 | tick_en=1 | row=000000110 | an=10111111 | seg=0100000 | dp=1
Time: 795 | data=aa | delay=1 | size=2 | tick_en=1 | row=000000110 | an=01111111 | seg=0000001 | dp=1
Time: 800 | data=aa | delay=1 | size=2 | tick_en=0 | row=000000111 | an=01111111 | seg=0000001 | dp=1
Time: 810 | data=aa | delay=1 | size=3 | tick_en=1 | row=000000111 | an=01111111 | seg=0000001 | dp=1
Time: 905 | data=aa | delay=1 | size=3 | tick_en=1 | row=000000111 | an=11111110 | seg=0001000 | dp=1
   ... (34 more lines: the second round with delay = 2, 3, data 0x55 with a reset, row 0FF / 100 / 000) ...
$finish called at time : 2305 ns : File "sim/tb_display_ctrl.v" Line 53
```

צורות הגל מוצגות באיור 23.



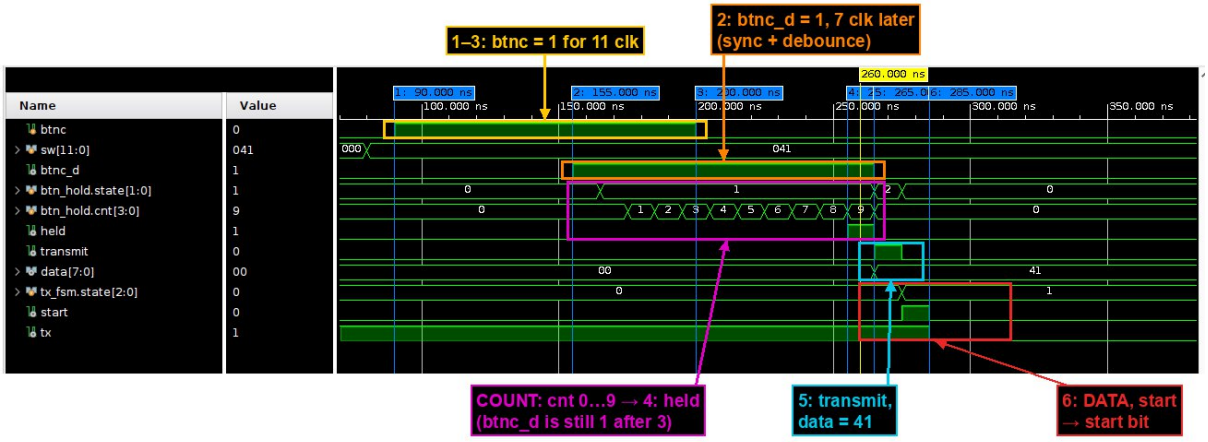
איור 23: tb_display_ctrl – כל הריצה: טבעת ה-an, המפענחים, הבורר ו-dp.

3.9. tb_chip_top_0 – בדיקת המערכת המלאה

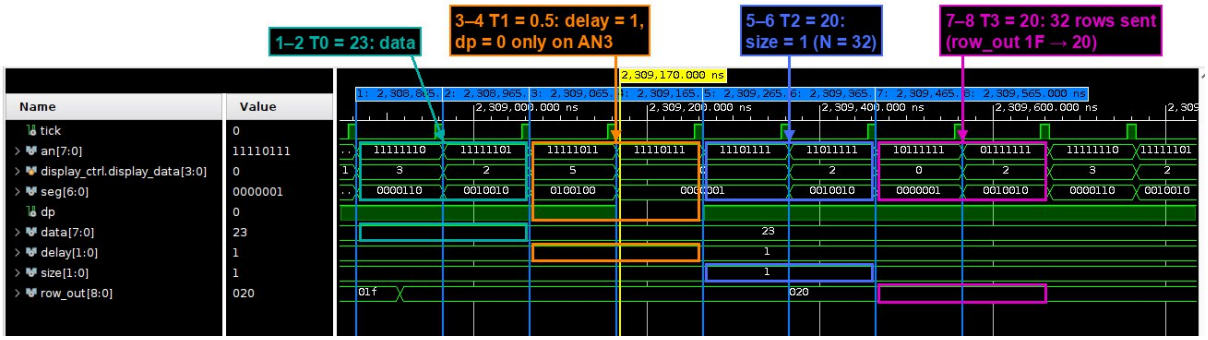
פלט הטסטבנץ' (tb_chip_top_0_xsim.log):

```
Time: 75 | data=0x41 | delay=0 | size=0
Time: 3315 | data=0x23 | delay=1 | size=1
$finish called at time : 2309925 ns : File "sim/tb_chip_top_0.v" Line 55
```

צורות הגל מוצגות באיורים 24–25.



איור 24: tb_chip_top_0 – זום: מהלחיצה ועד ה-start bit.



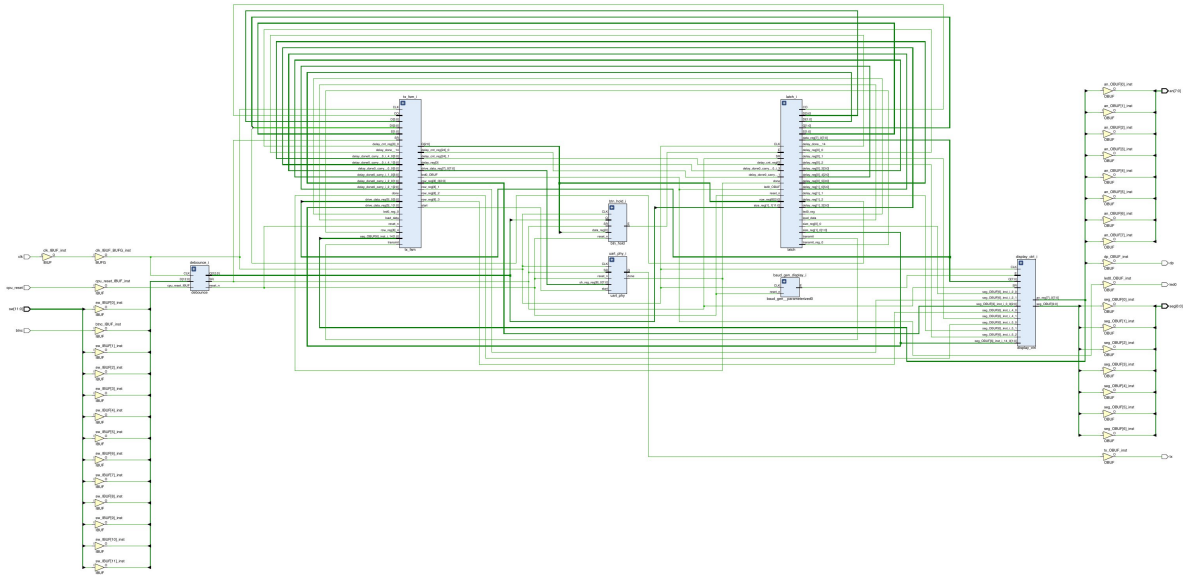
איור 25: tb_chip_top_0 – זום: סריקת התצוגה בסוף ריצת size=1, delay=1, SW=0x23, 32×32.

4. סינתזה ומימוש (Synthesis & Implementation)

4.1. סינתזה

טבלה 15: תוצאות הסינתזה ודוחות ה-Synthesized Design.

Report	Result
Messages (synthesis.log)	0 errors, 0 critical warnings, 0 warnings
Timing summary (timing_summary_synth.rpt)	sys_clk_pin 10.000 ns (100 MHz): WNS +2.096 ns, TNS 0.000, WHS +0.140 ns, THS 0.000, WPWS +4.500 ns, 0 failing of 636 endpoints – met
DRC (drc_synth.rpt)	0 checks
Methodology (methodology_synth.rpt)	0 checks
Clock networks / interaction	one clock: sys_clk_pin 100 MHz on clk (1 BUFG); only sys_clk_pin → sys_clk_pin paths, classification Clean
Noise (noise_synth.rpt)	18 output pins PASS, 0 FAIL
Power (power_synth.rpt)	0.101 W total on-chip (estimate)



איור 26: סכמת ה-post-synthesis של chip_top.

4.2. מימוש (Implementation)

טבלה 16: תוצאות המימוש ודוחות ה-Implemented Design.

Report	Result
Messages (implementation.log)	0 errors, 0 critical warnings, 0 warnings
Timing summary (timing_summary_routed.rpt)	WNS +2.264 ns, TNS 0.000 (0 of 663 endpoints failing); WHS +0.122 ns, THS 0.000; WPWS +4.500 ns – all constraints met
Critical path	latch_i/delay_reg[1]/C → tx_fsm_i/drive_data_reg[0]/CE: 7.376 ns, 9 logic levels (CARRY4=6 LUT2=1 LUT4=1 LUT6=1); logic 3.798 ns, route 3.578 ns
Route status (route_status.rpt)	682 logical nets, 432 routable, 432 fully routed, 0 with routing errors
DRC (drc_routed.rpt)	0 checks
Methodology (methodology_drc_routed.rpt)	0 checks
Power (power_routed.rpt)	0.100 W total: 0.097 W static + 0.003 W dynamic; junction 25.5 °C
I/O (io_placed.rpt)	all 33 ports placed on the XDC pins, LVCMOS33 (sw[8], sw[9] LVCMOS18)

5. ניצול משאבים

הערכים נלקחו מדוחות ה-utilization של העיצוב המלא.

טבלה 17: ניצול העיצוב המלא לאחר סינתזה ולאחר Place & Route.

Site type	Synthesis	Placed	Available	Util % (placed)
Slice LUTs (all as logic)	170	164	63,400	0.26
Slice Registers (FDRE + FDSE)	246	255	126,800	0.20
Register as Latch	0	0	126,800	0.00
Slices (placed: 62 SLICEL + 37 SLICEM)	–	99	15,850	0.62
F7 / F8 Muxes	0	0	31,700	0.00
Block RAM Tile	0	0	135	0.00
DSPs	0	0	240	0.00
Bonded IOB (15 inputs + 18 outputs)	33	33	210	15.71
BUFGCTRL	1	1	32	3.13
Unique control sets	17	17	15,850	0.11

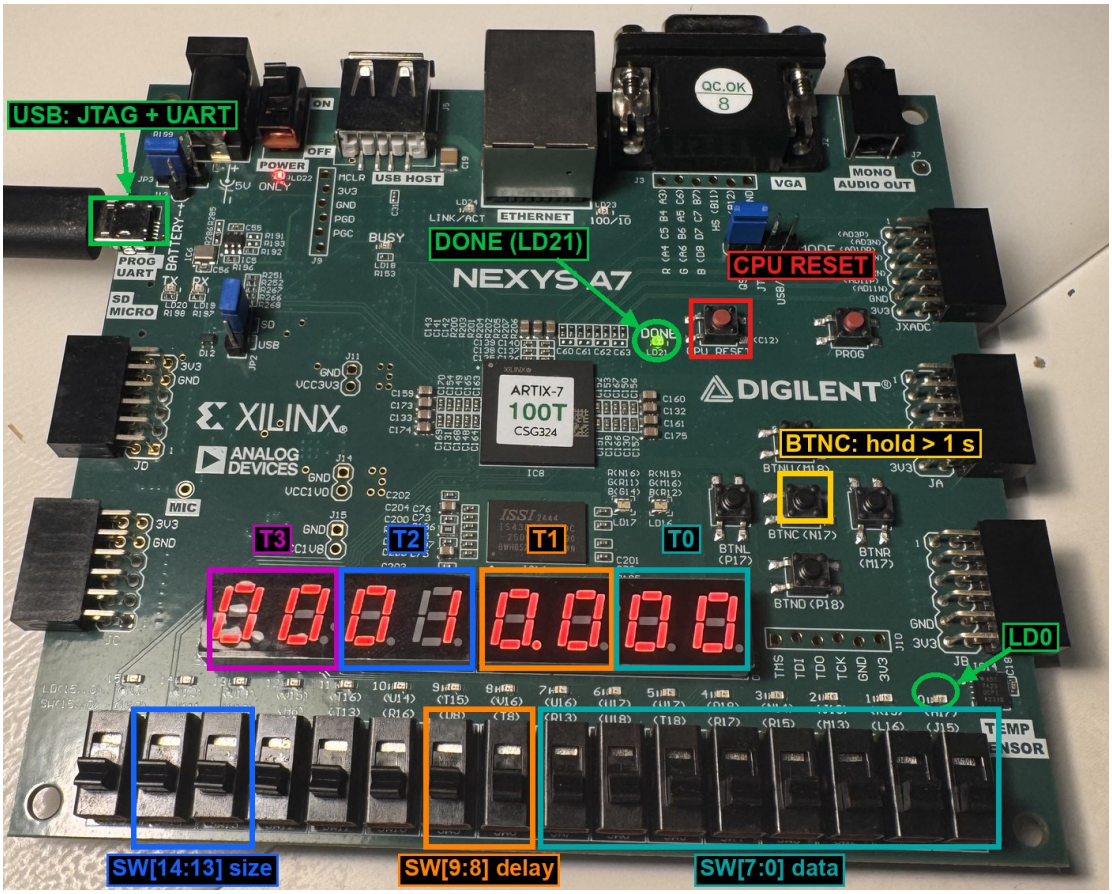
6. בדיקה על הלוח

6.1. יצירת ה-bitstream וטעינתו ללוח

טבלה 18: סיכום יצירת ה-bitstream.

Item	Result
Bitstream	chip_top.bit – 0 errors, 0 critical warnings, 0 warnings
Device	xc7a100tcsg324-1 (Digilent Nexys A7-100T)
Clock constraint	create_clock sys_clk_pin, period 10.000 ns on clk (E3)
Programming	JTAG over the on-board USB port; DONE LED on
Terminal	tio on macOS, 57600 bps, 8N1, no flow control (USB-UART of the board), log to output.txt

6.2. בדיקה פונקציונלית על הלוח



איור 27: הלוח לאחר טעינת ה-bitstream: נורית DONE דולקת והתצוגה במצב ההתחלתי.

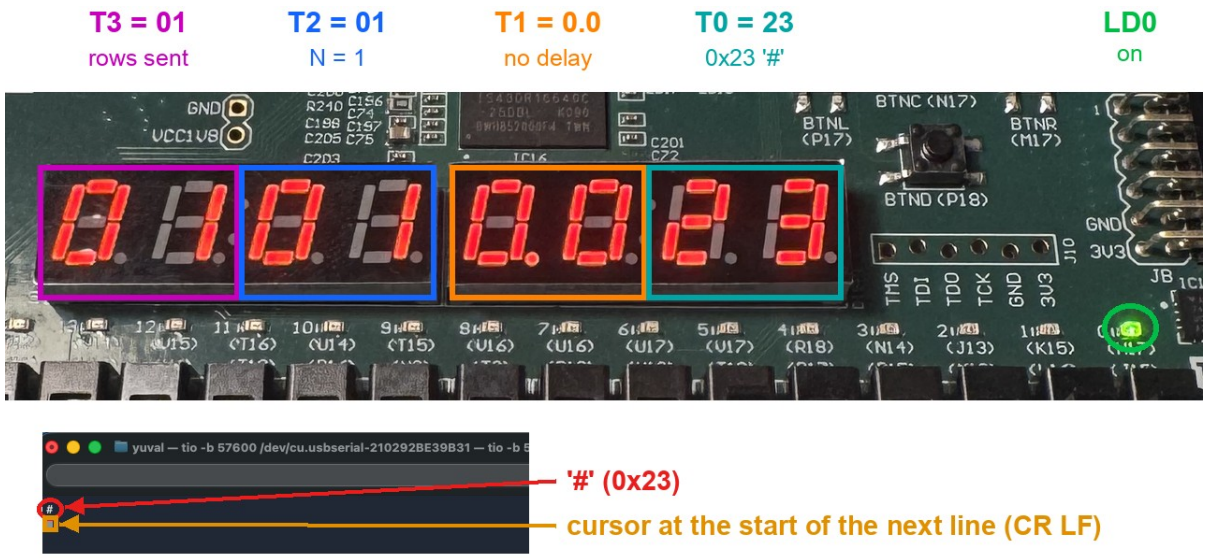
ב-macOS הטרמינל הופעל עם tio, שרושם גם כל תו שמתקבל לקובץ output.txt:

```
tio -b 57600 --log --log-file "$PWD/output.txt" --log-strip /dev/cu.usbserial-210292BE39B31
```

לפני כל מקרה הוגדרו המתגים, ואחר כך BTNC הוחזק יותר משנייה. הבית הנשלח הוא תמיד 0x23, התו '#'. בעמודת SW מופיעים SW[7:0] / SW[9:8] / SW[14:13], ובעמודת Display השדות T3 T2 T1 T0 משמאל לימין. טבלה 19 מרכזת את המקרים, ואיורים 31–28 מציגים אותם.

טבלה 19: מקרי הבדיקה על הלוח.

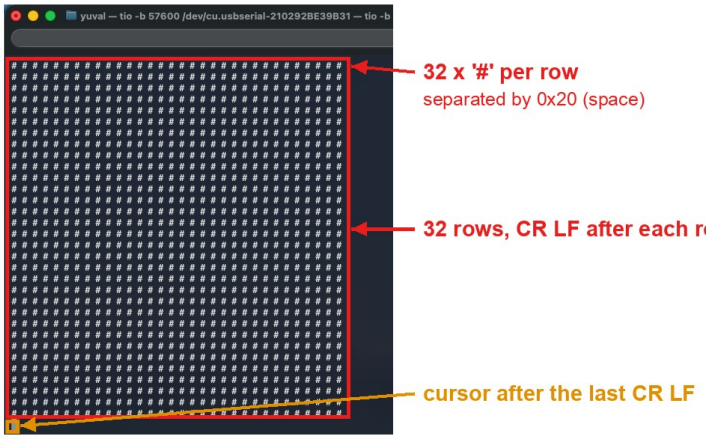
Case	SW	Button	Mode	Display	Result	Figure
1	00 / 00 / 0x23	BTNC > 1 s	1×1, 0 ms	01 01 0.0 23	Pass	28
2	01 / 00 / 0x23	BTNC > 1 s	32×32, 0 ms	20 20 0.0 23	Pass	29
3	11 / 00 / 0x23	BTNC > 1 s	256×256, 0 ms	29 FF 0.0 23 → FF FF 0.0 23	Pass	30
4	01 / 11 / 0x23	BTNC > 1 s	32×32, 200 ms	00 20 2.0 23 → 01 20 2.0 23	Pass	31



איור 28: מקרה 1, ריבוע 1×1: התצוגה ו-LD0 (למעלה) והטרמינל (למטה).

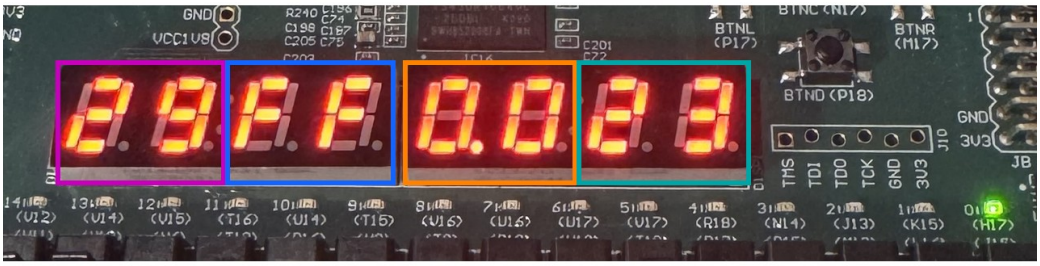
T3 = 20 32 rows sent | **T2 = 20** N = 32 | **T1 = 0.0** no delay | **T0 = 23** 0x23 '#'



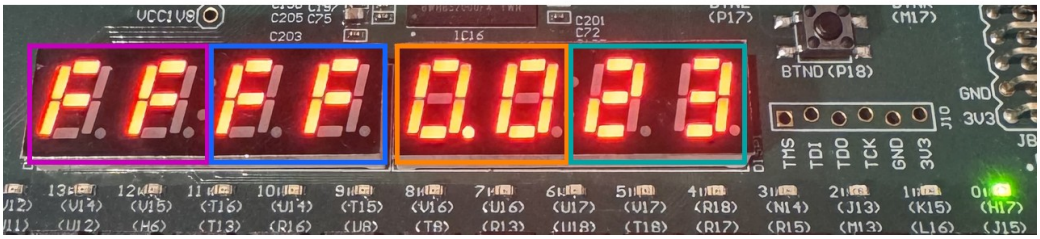


איור 29: מקרה 2, ריבוע 32×32: התצוגה (למעלה) והטרמינל (למטה).

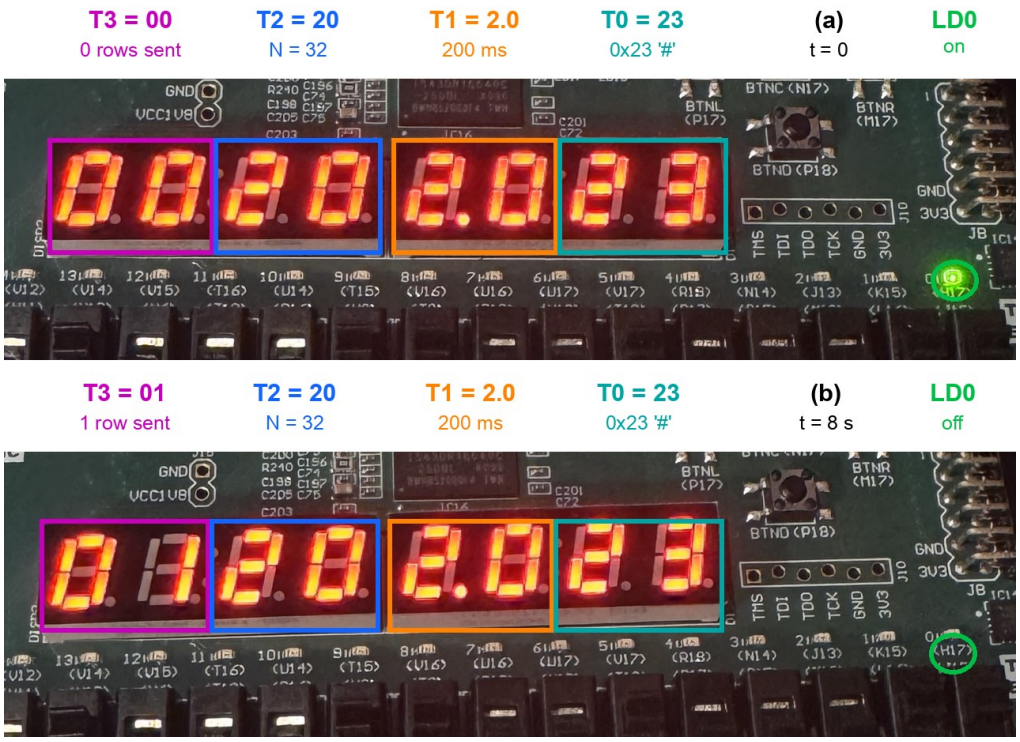
T3 = 29 41 rows sent | **T2 = FF** N = 256 | **T1 = 0.0** no delay | **T0 = 23** 0x23 '#' | **(a)** mid-run



T3 = FF 256 rows (sat.) | **T2 = FF** N = 256 | **T1 = 0.0** no delay | **T0 = 23** 0x23 '#' | **(b)** end



איור 30: מקרה 3, ריבוע 256×256: באמצע הריצה, (a) בסופה, (b).



איור 31: מקרה 4, ריבוע 32×32 עם השהיה של 200ms; תמונה (b) צולמה 8 שניות אחרי (a).

קובץ הלוג נרשם בהרצה נוספת אחרי הצילומים, ויש בו 6 ריצות: ריבועים של '#' בכל ארבעת הגדלים וריבועים של '$' (0x24) בגדלים 32 ו-128. בכל ריצה יש N שורות, וכל שורה מכילה בדיוק N עותקים של הבית עם רווח יחיד ביניהם וללא רווח בסוף השורה (טבלה 20). בקובץ אין תווי CR, כי האפשרות log-strip של tio מסננת תווי בקרה; בטרמינל עצמו כל שורה מתחילה בעמודה הראשונה, כלומר ה-CR התקבל.

טבלה 20: בדיקת קובץ הלוג output.txt.

Run	Byte	Rows × bytes per row	Data bytes	Result
1	0x23 '#'	1 × 1	1	Pass
2	0x23 '#'	32 × 32	1,024	Pass
3	0x23 '#'	128 × 128	16,384	Pass
4	0x23 '#'	256 × 256	65,536	Pass
5	0x24 '$'	32 × 32	1,024	Pass
6	0x24 '$'	128 × 128	16,384	Pass
Total			100,353	Pass

נספח א': קוד המקור (RTL ואילוצים)

chip_top.v

```
`timescale 1ns/1ps
module chip_top #(
    parameter BAUD_RATE = 57600,
    parameter DISPLAY_RR = 4000,
    parameter MAX_COUNT_DEBOUNCE = 2000000,
    parameter MAX_COUNT_BTN_HOLD = 100000000,
    parameter DELAY50 = 5000000
    ) (
    input clk, cpu_reset,
    input btnc,
    input [11:0] sw,
    output tx,
    output led0,
    output [7:0] an,
    output [6:0] seg,
    output dp
);

    wire [11:0] sw_d;
    wire btnc_d;
    wire reset_n;
    debounce #(.MAX_COUNT(MAX_COUNT_DEBOUNCE)) debounce_i (
        .clk(clk),
        .cpu_reset(cpu_reset),
        .btnc(btnc),
        .sw(sw),
        .sw_d(sw_d),
        .btnc_d(btnc_d),
        .cpu_reset_d(reset_n)
    );

    wire held;
    btn_hold #(.MAX_COUNT(MAX_COUNT_BTN_HOLD)) btn_hold_i (
        .clk(clk),
        .reset_n(reset_n),
        .btn(btnc_d),
        .held(held)
    );

    wire ready;
    wire [7:0] data;
    wire [1:0] delay, size;
    wire transmit;
    latch latch_i (
        .clk(clk),
        .reset_n(reset_n),
        .ready(ready),
        .sw_d(sw_d),
        .held(held),
        .data(data),
        .delay(delay),
        .size(size),
```

```
        .transmit(transmit)
    );

    wire done;
    wire [7:0] drive_data;
    wire start;
    wire [8:0] row_out;
    tx_fsm #(.DELAY50(DELAY50)) tx_fsm_i (
        .clk(clk),
        .reset_n(reset_n),
        .data(data),
        .delay(delay),
        .size(size),
        .transmit(transmit),
        .done(done),
        .drive_data(drive_data),
        .start(start),
        .led0(led0),
        .row_out(row_out),
        .ready(ready)
    );

    uart_phy #(.BAUD_RATE(BAUD_RATE)) uart_phy_i (
        .clk(clk),
        .reset_n(reset_n),
        .data(drive_data),
        .start(start),
        .tx(tx),
        .done(done)
    );

    wire tick;
    baud_gen #(.BAUD_RATE(DISPLAY_RR)) baud_gen_display_i (
        .clk(clk),
        .reset_n(reset_n),
        .en(1'b1),
        .baud_tick(tick)
    );

    display_ctrl display_ctrl_i (
        .clk(clk),
        .reset_n(reset_n),
        .tick(tick),
        .data(data),
        .delay(delay),
        .size(size),
        .row(row_out),
        .an(an),
        .seg(seg),
        .dp(dp)
    );

endmodule
```

debounce.v

```
`timescale 1ns/1ps
module debounce #(parameter MAX_COUNT = 2000000) (
    input clk,
    input cpu_reset,
    input btnc,
    input [11:0] sw,
    output [11:0] sw_d,
    output btnc_d,
    output cpu_reset_d
);

    localparam MAX_COUNT_BITS = $clog2(MAX_COUNT);
    localparam [MAX_COUNT_BITS-1:0] ONE = 1;
    // Debonce, need to wait for 20 ms -> 100M * 20m = 2M = MAX_COUNT
    // Going to use 2 counters, 1 for sw + btnc and 1 from reset (because reset is important buton)
    // 1. Need synchronizer for all the async inputs.
    // 2. Need to detect a change (dual edge detector)
    // 3. once change is detected, start counting, if stable for count == MAX_COUNT - 1 can latch the values to
outputs

    // I'll start with reset debounce so that i can use the reset signal as reset for the other debouncer
    // Synchronizer - reset (Initial valuse for reset regs so we start at reset automatically)
    reg reset_sync = 1'b0;
    reg reset_sync_0 = 1'b0;
    always @(posedge clk) begin
        {reset_sync, reset_sync_0} <= {reset_sync_0, cpu_reset};
    end

    // Edge detector - reset
    reg reset_prev = 1'b0;
    always @(posedge clk) begin
        reset_prev <= reset_sync;
    end
    wire reset_edge = reset_prev ^ reset_sync;

    // Counter - reset
    reg [MAX_COUNT_BITS-1:0] reset_cnt = {MAX_COUNT_BITS{1'b0}};
    wire reset_stable = (reset_cnt == MAX_COUNT - 1);
    always @(posedge clk) begin
        if (reset_edge) reset_cnt <= {MAX_COUNT_BITS{1'b0}};
        else if (!reset_stable) reset_cnt <= reset_cnt + ONE;
    end

    // Output reg - reset
    reg reset_n = 1'b0;
    always @(posedge clk) begin
        if (reset_stable & !reset_edge) reset_n <= reset_sync;
    end

    assign cpu_reset_d = reset_n;

    // Now SW + BTNC (Can use reset now)
    // 1. Synchronizer
    reg [12:0] sw_btn_sync, sw_btn_sync_0;
```

```
    always @(posedge clk) begin
        if (!cpu_reset_d) {sw_btn_sync, sw_btn_sync_0} <= {26{1'b0}};
        else {sw_btn_sync, sw_btn_sync_0} <= {sw_btn_sync_0, {sw, btnc}};
    end

    // 2. Edge detector
    reg [12:0] sw_btn_prev;
    always @(posedge clk) begin
        if (!cpu_reset_d) sw_btn_prev <= {13{1'b0}};
        else sw_btn_prev <= sw_btn_sync;
    end
    wire sw_btn_edge = |(sw_btn_prev ^ sw_btn_sync);

    // 3. Counter
    reg [MAX_COUNT_BITS-1:0] sw_btn_cnt;
    wire sw_btn_stable = sw_btn_cnt == MAX_COUNT - 1;
    always @(posedge clk) begin
        if (!cpu_reset_d | sw_btn_edge) sw_btn_cnt <= {MAX_COUNT_BITS{1'b0}};
        else if (!sw_btn_stable) sw_btn_cnt <= sw_btn_cnt + ONE;
    end

    // Output reg
    reg [12:0] sw_btn_d;
    always @(posedge clk) begin
        if (!cpu_reset_d) sw_btn_d <= {13{1'b0}};
        else if (sw_btn_stable & !sw_btn_edge) sw_btn_d <= sw_btn_sync;
    end

    assign sw_d = sw_btn_d[12:1];
    assign btnc_d = sw_btn_d[0];

endmodule
```

btn_hold.v

```
`timescale 1ns/1ps
// MAX_COUNT = 100M * 1 = 100M

module btn_hold #(parameter MAX_COUNT = 100000000) (
    input clk, reset_n,
    input btn,
    output held
);

    localparam MAX_COUNT_BITS = $clog2(MAX_COUNT);
    localparam [MAX_COUNT_BITS-1:0] ONE = 1;

    localparam IDLE = 0, COUNT = 1, WAIT = 2;
    reg [1:0] state, next;

    reg [MAX_COUNT_BITS-1:0] cnt;

    always @(posedge clk) begin
        if (!reset_n) state <= IDLE;
        else state <= next;
```

```
    end

    always @(*) begin
        next = state;
        case (state)
            IDLE: next = btn ? COUNT : IDLE;
            COUNT: begin
                if (!btn) next = IDLE;
                else next = (cnt == MAX_COUNT - 1) ? WAIT : COUNT;
            end
            WAIT: next = btn ? WAIT : IDLE;
            default: next = IDLE;
        endcase
    end


    always @(posedge clk) begin
        if (!reset_n | state != next) cnt <= {MAX_COUNT_BITS{1'b0}};
        else if (state == COUNT) cnt <= cnt + ONE;
    end

    assign held = (state == COUNT) & btn & (cnt == MAX_COUNT - 1);
endmodule
```

latch.v

```
`timescale 1ns/1ps
module latch (
    input clk, reset_n,
    input ready,
    input [11:0] sw_d,
    input held,
    output reg [7:0] data,
    output reg [1:0] delay,
    output reg [1:0] size,
    output reg transmit
);

    wire load_cfg = held & ready;

    // sw reg
    always @(posedge clk) begin
        if (!reset_n) {size, delay, data} <= 12'b0;
        else if (load_cfg) {size, delay, data} <= {sw_d[11:10], sw_d[9:8], sw_d[7:0]};
    end


    // transmit reg
    always @(posedge clk) begin
        if (!reset_n) transmit <= 1'b0;
        else transmit <= load_cfg;
    end

endmodule
```

tx_fsm.v

```
`timescale 1ns/1ps
module tx_fsm #(parameter DELAY50=5000000)(
    input clk, reset_n,
    input [7:0] data,
    input [1:0] delay,
    input [1:0] size,
    input transmit,
    input done,
    output reg [7:0] drive_data,
    output reg start,
    output reg led0,
    output [8:0] row_out,
    output ready
);

    // Delay Calculation:
    // 1. No delay no need to count anything
    // 2. 50ms -> 100M * 50m = 5,000,000
    // 3. 100ms -> 100M * 100m = 10,000,000
    // 4. 200ms -> 100M * 200m = 20,000,000
    localparam DELAY0=0, DELAY100=2*DELAY50, DELAY200=4*DELAY50;
    localparam DELAY_BITS = $clog2(DELAY200+1); // to cover all delay types i took max(DELAYS)
    localparam [DELAY_BITS-1:0] DELAY_ONE = 1;

    localparam SIZE0 = 1, SIZE1 = 32, SIZE2 = 128, SIZE3 = 256;
    localparam SIZE_BITS = $clog2(SIZE3);
    localparam [SIZE_BITS-1:0] SIZE_ONE = 1;

    localparam [7:0] SPACE_DATA = 8'h20, NEW_LINE_0_DATA = 8'h0D, NEW_LINE_1_DATA = 8'h0A;

    localparam IDLE = 0, DATA = 1, SPACE = 2, NEW_LINE_0 = 3, NEW_LINE_1 = 4, WAIT = 5;

    // Registers
    reg [2:0] state, next;
    reg [DELAY_BITS-1:0] delay_reg;
    reg [SIZE_BITS-1:0] size_reg;
    reg [DELAY_BITS-1:0] delay_cnt;
    reg [SIZE_BITS-1:0] col;
    reg [SIZE_BITS:0] row;

    //Decoders
    // 1. delay decodet:
    always @(*) begin
        case (delay)
            2'd0: delay_reg = DELAY0;
            2'd1: delay_reg = DELAY50;
            2'd2: delay_reg = DELAY100;
            2'd3: delay_reg = DELAY200;
        endcase
    end

    // 2. size decoder:
```

```
always @(*) begin
   case(size)
      2'd0: size_reg = SIZE0-1;
      2'd1: size_reg = SIZE1-1;
      2'd2: size_reg = SIZE2-1;
      2'd3: size_reg = SIZE3-1;
   endcase
end

// Signals
wire last_col = (col == size_reg);
wire last_row = (row == size_reg);

// Counters
wire delay_done = (delay_cnt == delay_reg - 1) | (delay_reg == DELAY0);
// signal do reset wait counter:
// 1. when coming from SPACE: state == SPACE & done
// 2. when coming from NEW_LINE1: state == NEW_LINE1 & done & !last_row;
wire rst_delay_count = ((state == SPACE) & done) | ((state == NEW_LINE_1) & done & !last_row);
always @(posedge clk) begin
   if (!reset_n | rst_delay_count) delay_cnt <= {DELAY_BITS{1'b0}};
   else if (!delay_done) delay_cnt <= delay_cnt + DELAY_ONE;
end

wire rst_counters = (state == IDLE) & transmit;
// col counter needs to +1 from transition from SPACE TO WAIT
wire col_transtition = (state == SPACE) & done & !last_col;
// row counter needs to up one from NEW_LINE_1 transition to WAIT
wire row_transtition = (state == NEW_LINE_1) & done;
always @(posedge clk) begin
   if (!reset_n | rst_counters | row_transtition) col <= {SIZE_BITS{1'b0}};
   else if (col_transtition) col <= col + SIZE_ONE;
end

always @(posedge clk) begin
   if (!reset_n | rst_counters) row <= {(SIZE_BITS+1){1'b0}};
   else if (row_transtition) row <= row + {1'b0, SIZE_ONE};
end

//FSM
always @(posedge clk) begin
   if (!reset_n) state <= IDLE;
   else state <= next;
end

always @(*) begin
   next = state;
   case (state)
      IDLE: next = transmit ? DATA : IDLE;
      DATA: if (done) next = (last_col) ? NEW_LINE_0 : SPACE;
      SPACE: next = (done) ? WAIT : SPACE;
      NEW_LINE_0: next = (done) ? NEW_LINE_1 : NEW_LINE_0;
      NEW_LINE_1: if (done) next = (last_row) ? IDLE : WAIT;
      WAIT: next = (delay_done) ? DATA : WAIT;
      default: next = state;
   endcase
```

```
    end

    // Registered outputs
    wire load_data = ((state == IDLE) & transmit) | ((state == WAIT) & delay_done);
    wire load_space= (state == DATA) & !last_col & done;
    wire load_new_line_0 = (state == DATA) & last_col & done;
    wire load_new_line_1 = (state == NEW_LINE_0) & done;
    always @(posedge clk) begin
        if (!reset_n) {start, drive_data, led0} <= {10{1'b0}};
        else begin
            start <= (load_data | load_space | load_new_line_0 | load_new_line_1);
            if (load_data) {drive_data, led0} <= {data, !led0};
            else if (load_space) drive_data <= SPACE_DATA;
            else if (load_new_line_0) drive_data <= NEW_LINE_0_DATA;
            else if (load_new_line_1) drive_data <= NEW_LINE_1_DATA;
        end
    end
    assign row_out = row;
    assign ready = (state == IDLE);
endmodule
```

uart_phy.v

```
`timescale 1ns/1ps
module uart_phy #(parameter BAUD_RATE = 57600) (
    input clk, reset_n,
    input [7:0] data,
    input start,
    output tx,
    output done
);

    // How it works:
    // 2 States, IDLE and TRANSMIT
    // tx is 1 all the time when not in transmit
    // to move to transmit state we need start (then when in transmit -> pass the state to the baud_gen as en so it starts ticking)
    // done is asserted when transmitted bits counter is 9 and there is a tick
    // Counter counter from 0-9 only in transmit and only when there is a tick to make sure that we space the bits in the correct timing
    // Shift register 10 bits, in IDLE we load it with {10{1'b1}} so line is always 1, and when start we load it with {1'b1, data, 1'b0},
    // then for every count enable is acts as a shift en too, and we shift right while inserting a one
    // {1'b1, sh_reg[9:1]}


    // States + state regs
    localparam IDLE = 0, TRANSMIT = 1;
    reg state, next;


    // tick generator
    wire tick;
    baud_gen #(.BAUD_RATE(BAUD_RATE)) baud_gen_i (clk, reset_n, (state == TRANSMIT), tick);
```

```
    // Counter + shift reg
    localparam MAX_COUNT = 10; // count 10 bits
    localparam MAX_COUNT_BITS = $clog2(MAX_COUNT);
    localparam [MAX_COUNT_BITS-1:0] ONE = 1;
    reg [MAX_COUNT_BITS-1:0] cnt;
    reg [MAX_COUNT-1:0] sh_reg;

    //signal we need:
    // 1. To move from IDLE to Transmit we need start, and it is input signal,
    // 2. To move from TRANSMIT to idle: when count is 9 and their is a tick we just transmitted the last bit
(stop bit) so can go back to IDLE
    // 3. shift and count signal, when in state TRANSMIT and tick;
    // 4. signal when to load, when we are in idle and start, and not in transmit
    assign done = (cnt == MAX_COUNT - 1) & tick;
    wire sh_cnt_en = (state == TRANSMIT) & tick;
    wire load = (state == IDLE) & start;

    // counter control
    // 1. needs to be reset to 0.
    // 2. counts only in transmit ans only when their is a tick else sits idle at 0
    always @(posedge clk) begin
        if (!reset_n | done | state == IDLE) cnt <= {MAX_COUNT_BITS{1'b0}};
        else if (sh_cnt_en) cnt <= cnt + ONE;
    end

    // Shift register control
    // 1. reset to ones
    // 2. at start load with data and start stop bit
    // 3. in sh_cnt_ent shift right and insert 1
    always @(posedge clk) begin
        if (!reset_n | done) sh_reg <= {MAX_COUNT{1'b1}};
        else if (load) sh_reg <= {1'b1, data, 1'b0};
        else if (sh_cnt_en) sh_reg <= {1'b1, sh_reg[9:1]};
    end

    //FSM
    always @(posedge clk) begin
        if (!reset_n) state <= IDLE;
        else state <= next;
    end

    always @(*) begin
        next = state;
        case(state)
            IDLE: next = start ? TRANSMIT : IDLE;
            TRANSMIT: next = done ? IDLE : TRANSMIT;
            default: next = state;
        endcase
    end

    // output
    assign tx = sh_reg[0];
endmodule
```

baud_gen.v

```
`timescale 1ns/1ps
module baud_gen #(parameter BAUD_RATE = 57600) (
   input clk, reset_n,
   input en,
   output baud_tick
);

   localparam CLK_FREQ = 100000000;
   localparam COUNT = CLK_FREQ / BAUD_RATE;
   localparam COUNT_BITS = $clog2(COUNT);
   localparam [COUNT_BITS-1:0] ONE = 1;

   reg [COUNT_BITS-1:0] tick_cnt;

   assign baud_tick = (tick_cnt == COUNT - 1);

   always @(posedge clk) begin
     if (!reset_n | baud_tick | !en) tick_cnt <= {COUNT_BITS{1'b0}};
   else tick_cnt <= tick_cnt + ONE;
   end

endmodule
```

display_ctrl.v

```
`timescale 1ns/1ps
module display_ctrl (
   input clk, reset_n,
   input tick,
   input [7:0] data,
   input [1:0] delay,
   input [1:0] size,
   input [8:0] row,
   output reg [7:0] an,
   output [6:0] seg,
   output dp
);

   reg [3:0] display_data;

   // Like in lab 3 need a ring shifter for an
   always @(posedge clk) begin
      if (!reset_n) an <= 8'hFE;
      else if (tick) an <= {an[6:0], an[7]};
   end

   // need to call seven_segment
   seven_segment ss_i (display_data, seg);

   reg [7:0] delay_decode;
   always @(*) begin
      case(delay)
         2'd0: delay_decode = 8'h00;
```

```
            2'd1: delay_decode = 8'h05;
            2'd2: delay_decode = 8'h10;
            2'd3: delay_decode = 8'h20;
        endcase
    end

    reg [7:0] size_decode;
    always @(*) begin
        case(size)
            2'd0: size_decode = 8'h01;
            2'd1: size_decode = 8'h20;
            2'd2: size_decode = 8'h80;
            2'd3: size_decode = 8'hFF;
        endcase
    end

    wire [7:0] row_decode = row[8] ? 8'hFF : row[7:0];

    always @(*) begin
        case(an)
            8'hFE: display_data = data[3:0];
            8'hFD: display_data = data[7:4];
            8'hFB: display_data = delay_decode[3:0];
            8'hF7: display_data = delay_decode[7:4];
            8'hEF: display_data = size_decode[3:0];
            8'hDF: display_data = size_decode[7:4];
            8'hBf: display_data = row_decode[3:0];
            8'h7F: display_data = row_decode[7:4];
            default: display_data = 4'h0;
        endcase
    end


    assign dp = an[3];

endmodule
```

seven_segment.v

```
`timescale 1ns/1ps
module seven_segment (
    input [3:0] dig_in,
    output reg [6:0] ss_dig
);

    always @(*) begin
        case (dig_in)
            4'h0: ss_dig = 7'b0000001;
            4'h1: ss_dig = 7'b1001111;
            4'h2: ss_dig = 7'b0010010;
            4'h3: ss_dig = 7'b0000110;
            4'h4: ss_dig = 7'b1001100;
            4'h5: ss_dig = 7'b0100100;
            4'h6: ss_dig = 7'b0100000;
            4'h7: ss_dig = 7'b0001111;
```

```
        4'h8: ss_dig = 7'b0000000;
        4'h9: ss_dig = 7'b0000100;
        4'hA: ss_dig = 7'b0001000;
        4'hB: ss_dig = 7'b1100000;
        4'hC: ss_dig = 7'b0110001;
        4'hD: ss_dig = 7'b1000010;
        4'hE: ss_dig = 7'b0110000;
        4'hF: ss_dig = 7'b0111000;
        default: ss_dig = 7'b1111111;
      endcase
  end

endmodule
```

NexysA7-100T_Master.xdc (האילוצים הפעילים בלבד)

```
set_property -dict { PACKAGE_PIN E3    IOSTANDARD LVCMOS33 } [get_ports { clk }]; #IO_L12P_T1_MRCC_35 Sch=clk100mhz
create_clock -add -name sys_clk_pin -period 10.00 -waveform {0 5} [get_ports {clk}];
set_false_path -from [get_ports {cpu_reset btnc sw[*]}]
set_false_path -to [get_ports {tx led0 an[*] seg[*] dp}]
set_property -dict { PACKAGE_PIN J15   IOSTANDARD LVCMOS33 } [get_ports { sw[0] }]; #IO_L24N_T3_RS0_15 Sch=sw[0]
set_property -dict { PACKAGE_PIN L16   IOSTANDARD LVCMOS33 } [get_ports { sw[1] }]; #IO_L3N_T0_DQS_EMCCLK_14 Sch=sw[1]
set_property -dict { PACKAGE_PIN M13   IOSTANDARD LVCMOS33 } [get_ports { sw[2] }]; #IO_L6N_T0_D08_VREF_14 Sch=sw[2]
set_property -dict { PACKAGE_PIN R15   IOSTANDARD LVCMOS33 } [get_ports { sw[3] }]; #IO_L13N_T2_MRCC_14 Sch=sw[3]
set_property -dict { PACKAGE_PIN R17   IOSTANDARD LVCMOS33 } [get_ports { sw[4] }]; #IO_L12N_T1_MRCC_14 Sch=sw[4]
set_property -dict { PACKAGE_PIN T18   IOSTANDARD LVCMOS33 } [get_ports { sw[5] }]; #IO_L7N_T1_D10_14 Sch=sw[5]
set_property -dict { PACKAGE_PIN U18   IOSTANDARD LVCMOS33 } [get_ports { sw[6] }]; #IO_L17N_T2_A13_D29_14 Sch=sw[6]
set_property -dict { PACKAGE_PIN R13   IOSTANDARD LVCMOS33 } [get_ports { sw[7] }]; #IO_L5N_T0_D07_14 Sch=sw[7]
set_property -dict { PACKAGE_PIN T8    IOSTANDARD LVCMOS18 } [get_ports { sw[8] }]; #IO_L24N_T3_34 Sch=sw[8]
set_property -dict { PACKAGE_PIN U8    IOSTANDARD LVCMOS18 } [get_ports { sw[9] }]; #IO_25_34 Sch=sw[9]
set_property -dict { PACKAGE_PIN U12   IOSTANDARD LVCMOS33 } [get_ports { sw[10] }]; #IO_L20P_T3_A08_D24_14 Sch=sw[13]
set_property -dict { PACKAGE_PIN U11   IOSTANDARD LVCMOS33 } [get_ports { sw[11] }]; #IO_L19N_T3_A09_D25_VREF_14 Sch=sw[14]
set_property -dict { PACKAGE_PIN H17   IOSTANDARD LVCMOS33 } [get_ports { led0 }]; #IO_L18P_T2_A24_15 Sch=led[0]
set_property -dict { PACKAGE_PIN T10   IOSTANDARD LVCMOS33 } [get_ports { seg[6] }]; #IO_L24N_T3_A00_D16_14 Sch=ca
set_property -dict { PACKAGE_PIN R10   IOSTANDARD LVCMOS33 } [get_ports { seg[5] }]; #IO_25_14 Sch=cb
set_property -dict { PACKAGE_PIN K16   IOSTANDARD LVCMOS33 } [get_ports { seg[4] }]; #IO_25_15 Sch=cc
set_property -dict { PACKAGE_PIN K13   IOSTANDARD LVCMOS33 } [get_ports { seg[3] }]; #IO_L17P_T2_A26_15 Sch=cd
set_property -dict { PACKAGE_PIN P15   IOSTANDARD LVCMOS33 } [get_ports { seg[2] }]; #IO_L13P_T2_MRCC_14 Sch=ce
```

```
set_property -dict { PACKAGE_PIN T11   IOSTANDARD LVCMOS33 } [get_ports { seg[1] }];
#IO_L19P_T3_A10_D26_14 Sch=cf
set_property -dict { PACKAGE_PIN L18   IOSTANDARD LVCMOS33 } [get_ports { seg[0] }]; #IO_L4P_T0_D04_14
Sch=cg
set_property -dict { PACKAGE_PIN H15   IOSTANDARD LVCMOS33 } [get_ports { dp }];
#IO_L19N_T3_A21_VREF_15 Sch=dp
set_property -dict { PACKAGE_PIN J17   IOSTANDARD LVCMOS33 } [get_ports { an[0] }];
#IO_L23P_T3_FOE_B_15 Sch=an[0]
set_property -dict { PACKAGE_PIN J18   IOSTANDARD LVCMOS33 } [get_ports { an[1] }];
#IO_L23N_T3_FWE_B_15 Sch=an[1]
set_property -dict { PACKAGE_PIN T9    IOSTANDARD LVCMOS33 } [get_ports { an[2] }];
#IO_L24P_T3_A01_D17_14 Sch=an[2]
set_property -dict { PACKAGE_PIN J14   IOSTANDARD LVCMOS33 } [get_ports { an[3] }]; #IO_L19P_T3_A22_15
Sch=an[3]
set_property -dict { PACKAGE_PIN P14   IOSTANDARD LVCMOS33 } [get_ports { an[4] }]; #IO_L8N_T1_D12_14
Sch=an[4]
set_property -dict { PACKAGE_PIN T14   IOSTANDARD LVCMOS33 } [get_ports { an[5] }];
#IO_L14P_T2_SRCC_14 Sch=an[5]
set_property -dict { PACKAGE_PIN K2    IOSTANDARD LVCMOS33 } [get_ports { an[6] }]; #IO_L23P_T3_35
Sch=an[6]
set_property -dict { PACKAGE_PIN U13   IOSTANDARD LVCMOS33 } [get_ports { an[7] }];
#IO_L23N_T3_A02_D18_14 Sch=an[7]
set_property -dict { PACKAGE_PIN C12   IOSTANDARD LVCMOS33 } [get_ports { cpu_reset }];
#IO_L3P_T0_DQS_AD1P_15 Sch=cpu_resetn
set_property -dict { PACKAGE_PIN N17   IOSTANDARD LVCMOS33 } [get_ports { btnc }]; #IO_L9P_T1_DQS_14
Sch=btnc
set_property -dict { PACKAGE_PIN D4    IOSTANDARD LVCMOS33 } [get_ports { tx }]; #IO_L11N_T1_SRCC_35
Sch=uart_rxd_out
set_property CFGBVS VCCO [current_design]
set_property CONFIG_VOLTAGE 3.3 [current_design]
```

נספח ב': קוד הטסטבנצ'ים

הטסטבנצ'ים לפי הסדר של פרק 3:

tb_debounce.v

```
`timescale 1ns/1ps
module tb_debounce;

   localparam MAX_COUNT = 10;

   reg clk = 1'b0;
   always #5 clk = !clk;

   reg cpu_reset, btnc;
   reg [11:0] sw;
   wire [11:0] sw_d;
   wire btnc_d;
   wire cpu_reset_d;

   debounce #(.MAX_COUNT(MAX_COUNT)) debounce_i (clk, cpu_reset, btnc, sw, sw_d, btnc_d, cpu_reset_d);

   initial $monitor("Time: %0d | cpu_reset=%0b | cpu_reset_d=%0b | btnc=%0b | btnc_d=%0b | sw=%0h|
sw_d=%0h", $time, cpu_reset, cpu_reset_d, btnc, btnc_d, sw, sw_d);

   initial begin
      // power on reset wait for sync + edge to finish;
      cpu_reset = 1'b1; sw = 12'b0; btnc=1'b0;

      while (!debounce_i.reset_n == 1'b1) @(posedge clk);

      // test sw change
      @(negedge clk); sw = 12'hABC;
      repeat(MAX_COUNT + 3) @(posedge clk);

      // test switch dont change
      @(negedge clk) sw = 12'b0;
      repeat(MAX_COUNT - 1) @(posedge clk);
      @(negedge clk); sw = 12'hABC;

      //switch back to 0 and test btnc
      @(negedge clk); sw = 12'b0; btnc = 1'b1;
      repeat(MAX_COUNT + 3) @(posedge clk); // Here sw change back to zero
      @(negedge clk); btnc = 1'b0;

      // test if btnc doesnt change because wait not long enough
      repeat(MAX_COUNT -1) @(posedge clk);
      @(negedge clk); btnc = 1'b1;

      // cheack reset works in middle of change
      @(negedge clk); btnc = 1'b0; sw = 12'hABA;
      @(negedge clk); cpu_reset = 1'b0;
      repeat(MAX_COUNT + 3) @(posedge clk);
```

```
    @(negedge clk); cpu_reset = 1'b1;
    repeat(5) @(posedge clk);



    $finish;


  end

endmodule
```

tb_btn_hold.v

```
`timescale 1ns/1ps
module tb_btn_hold;

  localparam MAX_COUNT = 10;

  reg clk = 1'b0;
  always #5 clk = !clk;

  reg reset_n;
  reg btn;
  wire held;

  btn_hold #(.MAX_COUNT(MAX_COUNT)) btn_hold_i (clk, reset_n, btn, held);

  initial $monitor("Time: %0d | btn=%0b | held=%0b", $time, btn, held);

  initial begin
    reset_n = 1'b0; btn = 1'b0;
    repeat(2) @(posedge clk);

    @(negedge clk); reset_n = 1'b1;
    @(negedge clk); btn = 1'b1;
    repeat(MAX_COUNT + 5) @(posedge clk);
    @(negedge clk) btn = 1'b0;

    @(negedge clk); btn = 1'b1;
    repeat(MAX_COUNT - 2) @(posedge clk);
    @(negedge clk); btn = 1'b0;

    repeat(10) @(posedge clk);
    $finish;
  end



endmodule
```

tb_latch.v

```
`timescale 1ns/1ps
module tb_latch;

    reg clk = 1'b0;
    always #5 clk = !clk;

    reg reset_n, ready;
    reg [11:0] sw_d;
    reg held;

    wire [7:0] data;
    wire [1:0] delay, size;
    wire transmit;

    latch latch_i(clk, reset_n, ready, sw_d, held, data, delay, size, transmit);

    // held comes from btn_hold

    initial $monitor("Time: %0d | held=%0b | sw_d=%0h | ready=%0b | data=%0h | delay=%0d | size=%0d |
transmit=%0b", $time, held, sw_d, ready, data, delay, size, transmit);

    initial begin
        reset_n = 1'b0; ready = 1'b0; sw_d = 12'h000; held = 1'b0;
        repeat(2) @(posedge clk);
        @(negedge clk) reset_n = 1'b1;

        @(negedge clk); sw_d=12'hABC; held = 1'b1; ready = 1'b1;
        @(negedge clk); held = 1'b0; ready = 1'b0;

        @(negedge clk); sw_d=12'hABA; ready = 1'b1;
        @(negedge clk); sw_d=12'hFFF;
        @(negedge clk); sw_d=12'h555; held = 1'b1;
        @(negedge clk); ready = 1'b0;

        @(negedge clk); sw_d = 12'h111; held = 1'b0;
        @(negedge clk); held = 1'b0;
        @(negedge clk); held = 1'b1;
        @(negedge clk); ready = 1'b1;

        repeat(5) @(posedge clk);

        $finish;


    end

endmodule
```

tb_tx_fsm_0.v

```
`timescale 1ns/1ps
module tb_tx_fsm_0;

    reg clk = 1'b0;
    always #5 clk = !clk;

    reg reset_n;
    reg [7:0] data;
    reg [1:0] size, delay;
    reg transmit;
    wire done;
    wire [7:0] drive_data;
    wire start;
    wire led0;
    wire [8:0] row_out;
    wire ready;
    wire tx;

    uart_phy #(.BAUD_RATE(10000000)) uart_phy_i (clk, reset_n, drive_data, start, tx, done);
    tx_fsm #(.DELAY50(10)) tx_fsm_i(clk, reset_n, data, delay, size, transmit, done, drive_data, start, led0,
row_out, ready);

    initial begin
        reset_n = 1'b0; data = 8'b0; size = 2'b0; delay = 2'b0; transmit = 1'b0;
        repeat(2) @(posedge clk);

        reset_n = 1'b1;

        // 1. data = 0x41 delay 0 size 0
        $display("Time: %0d | data=0x41 | size=0 | delay=0", $time);
        @(negedge clk); data = 8'h41; transmit = 1'b1;
        @(negedge clk); transmit = 1'b0;
        while (!ready) @(posedge clk);

        // 2. delay 1, size 1, data=0x55
        $display("Time: %0d | data=0x55 | delay=1 | size=1", $time);
        @(negedge clk); data = 8'h55; delay =2'd1; size = 2'd1; transmit = 1'b1;
        @(negedge clk); transmit = 1'b0;
        while (!ready) @(posedge clk);

        // 3. delay 3, size 1, data=0xAA
        $display("Time: %0d | data=0xAA | delay=3 | size=1", $time);
        @(negedge clk); data = 8'hAA; delay =2'd3; size = 2'd1; transmit = 1'b1;
        @(negedge clk); transmit = 1'b0;
        while (!ready) @(posedge clk);

        $finish;

    end

endmodule
```

tb_uart_phy_0.v

```
`timescale 1ns/1ps
module tb_uart_phy_0;

localparam BAUD_RATE = 10000000; // 100M / 10M = 10 cycles per bit
localparam CYCLES_PER_BIT = 100000000 / BAUD_RATE; // = 10
localparam FRAME_CYCLES = CYCLES_PER_BIT * 10 + 10;

reg clk = 1'b0;
always #5 clk = !clk;

reg reset_n;
reg [7:0] data;
reg start;

wire tx;
wire done;


uart_phy #(.BAUD_RATE(BAUD_RATE)) uart_phy_i (clk, reset_n, data, start, tx, done);

initial $monitor("Time: %0d | data=%0h | start=%0b | tx=%0b | done=%0b", $time, data, start, tx, done);

initial begin
   reset_n = 1'b0; data =8'b0; start = 1'b0;
   repeat(2) @(negedge clk);
   @(negedge clk); reset_n = 1'b1;

   // Drive 0xA5
   @(negedge clk); data = 8'hA5; start = 1'b1;
   @(negedge clk); start = 1'b0;

   repeat(FRAME_CYCLES) @(posedge clk);

   // Drive 0x00
   @(negedge clk); data = 8'h00; start = 1'b1;
   @(negedge clk); start = 1'b0;

   repeat(FRAME_CYCLES) @(posedge clk);

   // Drive 0xFF
   @(negedge clk); data = 8'hFF; start = 1'b1;
   @(negedge clk); start = 1'b0;

   repeat(FRAME_CYCLES) @(posedge clk);

   $finish;
end


endmodule
```

tb_baud_gen.v

```
`timescale 1ns/1ps
module tb_baud_gen;

    localparam BAUD_RATE = 10000000;

    reg clk = 1'b0;
    always #5 clk = !clk;

    reg reset_n;
    reg en;
    wire baud_tick;

    baud_gen #(.BAUD_RATE(BAUD_RATE)) baud_gen_i(clk, reset_n, en, baud_tick);

    initial $monitor("Time: %0d | en=%0b | baud_tick=%0b", $time, en, baud_tick);

    initial begin
        reset_n = 1'b0; en = 1'b0;
        repeat(2) @(posedge clk);

        @(negedge clk); reset_n = 1'b1; en = 1'b1;
        repeat(20) @(posedge clk);

        @(negedge clk); en = 1'b0;
        repeat(10) @(posedge clk);

        $finish;
    end

endmodule
```

tb_seven_segment.v

```
`timescale 1ns/1ps

module tb_seven_segment;
   reg clk = 1'b0;
   always #5 clk = !clk;

   reg [3:0] dig_in;
   wire [6:0] ss_dig;

   seven_segment seven_segment_i (dig_in, ss_dig);

   integer i;

   initial $monitor("Time: %0d | dig_in=%0h | ss_dig = %b", $time, dig_in, ss_dig);

   initial begin
      for (i = 0; i < 16; i = i + 1) begin
         dig_in = i;
         repeat(2) @(posedge clk);
      end
      $finish;
   end


endmodule
```

tb_display_ctrl.v

```
`timescale 1ns/1ps
module tb_display_ctrl;

   reg clk = 1'b0;
   always #5 clk = !clk;

   reg reset_n;
   wire tick;
   reg [7:0] data;
   reg [1:0] delay, size;
   reg [8:0] row;
   wire [7:0]an;
   wire [6:0] seg;
   wire dp;
   reg tick_en;

   baud_gen #(.BAUD_RATE(10000000)) baud_gen_i(clk, reset_n, tick_en, tick);
   display_ctrl display_ctrl_i(clk, reset_n, tick, data, delay, size, row, an, seg, dp);

   integer i, j, k;

   initial $monitor("Time: %0d | data=%h | delay=%d | size=%d | tick_en=%0b | row=%b | an=%b | seg=%b
| dp=%0b", $time, data, delay, size, tick_en, row, an, seg, dp);
```

```
    initial begin
        reset_n = 1'b0; data = 8'h00; delay=2'd0; size = 2'd0; tick_en = 1'b0; row=9'd0;
        repeat(2) @(posedge clk);
        @(negedge clk); reset_n = 1'b1;

        @(negedge clk); data = 8'hAA;
        for (i = 0; i < 4; i = i + 1) begin
            for (j = 0; j < 4; j = j + 1) begin
                @(negedge clk); delay = i[1:0]; size = j[1:0]; tick_en = 1'b1;
                repeat(10) @(posedge clk);
                @(negedge clk) tick_en = 1'b0; row = row + 1;
            end
        end

        @(negedge clk); data = 8'h55; delay = 2'd0; size = 2'd0; tick_en = 1'b1;
        repeat(5) @(posedge clk);
        @(negedge clk); reset_n = 1'b0; row = 9'd0;
        @(negedge clk); reset_n = 1'b1;
        repeat(10) @(posedge clk);

        @(negedge clk); row = 9'h0ff;
        repeat(10) @(posedge clk);
        @(negedge clk); row = 9'h100;
        repeat(10) @(posedge clk);
        @(negedge clk); row = 9'h000;
        repeat(15) @(posedge clk);



        $finish;
    end


endmodule
```

tb_chip_top_0.v

```
`timescale 1ns/1ps
module tb_chip_top_0;

localparam BAUD_RATE = 10000000; // 100M / 10M = 10 cycles per bit
localparam DISPLAY_RR = 10000000; // tick every 10 cycles
localparam MAX_COUNT_DEBOUNCE = 4;
localparam MAX_COUNT_BTN_HOLD = 10; // 10 cycles = 100ns
localparam DELAY50 = 20;

localparam LONG_HOLD = MAX_COUNT_BTN_HOLD + 1;

reg clk = 1'b0;
always #5 clk = !clk;
```

```
reg cpu_reset;
reg btnc;
reg [11:0] sw;

wire tx;
wire led0;
wire [7:0] an;
wire [6:0] seg;
wire dp;


chip_top
#(.BAUD_RATE(BAUD_RATE), .DISPLAY_RR(DISPLAY_RR), .MAX_COUNT_DEBOUNCE(MAX_COUNT_DEBOUNCE
), .MAX_COUNT_BTN_HOLD(MAX_COUNT_BTN_HOLD), .DELAY50(DELAY50)) chip_top_i (clk, cpu_reset, btnc,
sw, tx, led0, an, seg, dp);


initial begin
   cpu_reset = 1'b1; btnc = 1'b0; sw = 12'b0;
   while (chip_top_i.reset_n !== 1'b1) @(posedge clk); // wait for the power-on reset

   // 1. data = 0x41 delay 0 size 0
   $display("Time: %0d | data=0x41 | delay=0 | size=0", $time);
   @(negedge clk); sw = {2'd0, 2'd0, 8'h41};
   @(negedge clk); btnc = 1'b1;
   repeat(LONG_HOLD) @(posedge clk);
   @(negedge clk); btnc = 1'b0;

   while (chip_top_i.ready) @(posedge clk); // run started
   while (!chip_top_i.ready) @(posedge clk); // run ended

   // 2. data = 0x23 delay 1 size 1
   $display("Time: %0d | data=0x23 | delay=1 | size=1", $time);
   @(negedge clk); sw = {2'd1, 2'd1, 8'h23};
   @(negedge clk); btnc = 1'b1;
   repeat(LONG_HOLD) @(posedge clk);
   @(negedge clk); btnc = 1'b0;

   while (chip_top_i.ready) @(posedge clk);
   while (!chip_top_i.ready) @(posedge clk);

   repeat(100) @(posedge clk);

   $finish;
end


endmodule
```